Универзитет у Крагујевцу

Bogdan Babaev

Forecasting Time-Varying Intermarket Dependencies Between Cryptocurrencies and Conventional Assets Using Machine Learning

Мастер рад

Крагујевац, 2026.

Универзитет у Крагујевцу



Назив студијског програма: ВЕШТАЧКА ИНТЕЛИГЕНЦИЈА

Ниво студија: Мастер академске студије

Предмет: Напредно машинско учење

Број индекса: 1ВИ/2023

Bogdan Babaev

Forecasting Time-Varying Intermarket Dependencies Between Cryptocurrencies and Conventional Assets Using Machine Learning

Мастер рад

Комисија за преглед и одбрану:

1. ванр. проф. др Владимир М. Миловановић - ментор
2. др Александар Пеулић, редовни професор
3. др Јасна Радуловић, редовни професор
4. др Милан Чабаркапа, доцент
5. др Иван Крстић, ванредни професор

Датум одбране: _____

Оцена: _____

У оквиру овог мастер рада кандидат треба да истражи могућност прогнозирања временски-променљивих међутржишних зависности између криптовалута и конвенционалних финансијских актива применом модела машинског учења. У склопу тога кандидат треба да:

1. прикупи историјске дневне цене за Bitcoin и традиционалне активе (акцијске индексе, фондове племенитих метала, индекс америчког долара, Ethereum) и изврши одговарајућу предобраду и синхронизацију скупа података;
2. развије репродуцибилан рачунарски поступак за израчунавање временски-променљивих корелационих циљева помоћу клизних прозора различитих дужина и конструкцију предиктивних особина заснованих на инерцији зависности, волатилности и повратима;
3. реализује ванузорачну евалуацију и поређење неколико модела машинског учења (AR, ElasticNet, Ridge, Random Forest, GBM, XGBoost) са DCC-GARCH(1,1) економетријским репером применом процедуре проклизавајућег прозора и Diebold-Mariano теста статистичке значајности;
4. уведе практични сигнални слој за рано детектовање стресних дана на тржишту традиционалних актива на основу динамике криптовалута и прогноза зависности, и вреднује га метрикама класификационе тачности прикладним за неуравнотежен скуп класа.

Препоручена литература:

[1] Robert F. Engle, „Dynamic Conditional Correlation: A Simple Class of Multivariate Generalized Autoregressive Conditional Heteroskedasticity Models", у: Journal of Business & Economic Statistics 20.3 (2002.), стр. 339–350.

[2] Francis X. Diebold и Roberto S. Mariano, „Comparing Predictive Accuracy", у: Journal of Business & Economic Statistics 13.3 (1995.), стр. 253–263.

[3] Hui Zou и Trevor Hastie, „Regularization and Variable Selection via the Elastic Net", у: Journal of the Royal Statistical Society: Series B 67.2 (2005.), стр. 301–320.

[4] Jerome H. Friedman, „Greedy Function Approximation: A Gradient Boosting Machine", у: The Annals of Statistics 29.5 (2001.), стр. 1189–1232.

[5] Tianqi Chen и Carlos Guestrin, „XGBoost: A Scalable Tree Boosting System", у: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016., стр. 785–794.

Крагујевац, 19. 5. 2026.

Ментор:

Др Владимир М. Миловановић,
ванредни професор

_____

Abstract

This thesis examines whether the dynamics of cryptocurrency markets contain usable information about how their dependence on conventional assets evolves over time, and whether such information can be converted into an early-warning signal for investors. Bitcoin is used as the base cryptocurrency and Ethereum as a second cryptocurrency reference, while the conventional universe comprises equity indices, precious-metal ETFs, and the U.S. dollar index. A reproducible machine-learning pipeline performs data acquisition, return construction, computation of rolling-correlation targets, feature engineering, and walk-forward evaluation against a leakage-safe DCC-GARCH benchmark. The results indicate that intermarket dependency is forecastable out of sample for every asset pair and window length, although almost all of the predictability is attributable to the persistence of the dependency series itself. Ridge regression, AR(1), and the HAR model form a near-indistinguishable leading cluster, each relying on this persistence through a different mechanism while attaining essentially the same accuracy; Ridge attains the lowest average RMSE, 0.0656. Every other model outperforms the DCC-GARCH benchmark by a clear margin. A second-stage investor signal, constructed from cryptocurrency features and the dependency forecasts, achieves modest, mostly above-chance performance; it functions as a supplementary risk-management input, not a standalone market-timing tool.

Keywords: cryptocurrencies, Bitcoin, intermarket dependency, correlation, machine learning, DCC-GARCH, forecasting, risk management.

Резиме

У овом мастер раду разматра се проблем прогнозирања променљивих међутржишних зависности између криптовалута и традиционалних финансијских актива. У фокусу је Bitcoin као основна криптовалута, док скуп традиционалних актива обухвата акцијске индексе, фондове везане за племените метале и индекс америчког долара, уз Ethereum као додатну криптовалуту за поређење. Развијен је репродуцибилан рачунарски поступак који покрива преузимање података, израчунавање логаритамских приноса, формирање временски променљивих корелационих циљева, конструисање особина, ванузорачну процену више модела машинског учења и поређење са DCC-GARCH економетријским репером. Поред основног задатка прогнозирања зависности, уведен је и практични слој за формирање раног сигнала ризика који треба да упозори инвеститора на стресне дане на тржишту традиционалних актива. Резултати показују да је временски променљива међутржишна зависност ванузорачно предвидива, при чему Ridge регресија, AR(1) и HAR модел формирају практично неразлучиву групу на врху рангирања — свако од њих користи аутокорелациону структуру циља на другачији начин, а Ridge постиже најнижи просечни RMSE од 0.0656. Сви остали модели значајно надмашују DCC-GARCH репер. Практични сигнални слој даје умерене резултате и има смисла пре свега као додатни филтар ризика, а не као самосталан механизам за свакодневно одређивање смера тржишта.

Кључне речи: криптовалуте, Bitcoin, међутржишне зависности, корелација, машинско учење, DCC-GARCH, управљање ризиком.

Contents

List of Abbreviations **8**

1 Introduction **9**
1.1 Motivation and research context 9
1.2 Research questions 9
1.3 Contributions of the thesis 9
1.4 Thesis outline 10

2 Theoretical Background and Literature Review **11**
2.1 Why intermarket dependency matters 11
2.2 Cryptocurrencies and conventional assets 11
2.3 Rolling correlation as a forecasting target 11
2.4 Econometric benchmark: DCC-GARCH 12
2.5 Machine-learning approaches to forecasting 13
2.5.1 The HAR model and persistence-based forecasting 13
2.5.2 Regularized linear models 14
2.5.3 Tree-based ensemble models 14
2.6 Walk-forward evaluation and information leakage 14
2.7 Model comparison and the Diebold–Mariano test 15
2.8 From dependency forecasting to investor signaling 15
2.9 Positioning of the present thesis 16

3 Methodology **17**
3.1 Research design 17
3.2 Asset universe and data source 17
3.3 Return construction and preprocessing 18
3.4 Target construction: rolling dependency 18
3.5 Feature engineering 19
3.6 Forecasting models 21
3.6.1 Baseline and autoregressive models 21
3.6.2 Regularized linear models 21
3.6.3 Tree ensembles and adaptive ensemble 21
3.6.4 DCC-GARCH benchmark 22
3.7 Investor signal layer 22
3.8 Walk-forward estimation framework 23
3.9 Evaluation metrics 23
3.10 Model comparison methodology 23
3.11 Implementation and reproducibility 24
3.12 Methodological expectations 24

4 Empirical Results **26**
4.1 Overview of the empirical output 26
4.2 Average dependency-forecasting performance 26
4.3 Representative forecast example 27
4.4 Forecasting performance by asset class 28
4.5 The persistence dominance result 28
4.6 Comparison with DCC-GARCH 29

4.7 Error-profile diagnostics . . . 30
4.8 Investor signal layer: average results . . . 31
4.9 Best signal configurations . . . 31
4.10 Practical interpretation of signal-layer performance . . . 32
4.11 Sensitivity to rolling window length . . . 33
4.12 Summary of empirical findings . . . 33

5 Discussion **35**
5.1 What the results actually say . . . 35
5.2 Why simple models dominate, and what that actually means . . . 35
5.3 ML versus DCC-GARCH: a fairer comparison . . . 35
5.4 The investor signal: what works and what does not . . . 36
5.5 Methodological choices that mattered . . . 36
5.6 What the results do not say . . . 37
5.7 Iterative model development: what changed across versions . . . 37
5.8 Limitations . . . 39

6 Conclusion **40**

A Appendix A: Data Description and Additional Background **41**
A.1 Descriptive role of the asset universe . . . 41
A.2 Visual overview of the dataset . . . 41
A.3 Dataset validation and quality diagnostics . . . 43
A.4 Fisher-z transformation of the correlation target . . . 44
A.5 Why multiple rolling windows are used . . . 45
A.6 Additional implementation remarks . . . 45
A.7 Summary . . . 45

B Appendix B: Supplementary Forecasting Results **46**
B.1 Model-comparison figures . . . 46
B.2 Hyperparameter and feature diagnostics . . . 47
B.3 Diagnostic plots for the representative equity pair . . . 48
B.4 Sensitivity by asset pair . . . 49
B.5 Interpretation . . . 50

C Appendix C: Supplementary Signal-Layer Results **51**
C.1 Representative signal output . . . 51
C.2 Cross-market warning interpretation . . . 51
C.3 Summary table . . . 52

D Appendix D: Reproducibility and Code Structure **53**
D.1 Project organization . . . 53
D.2 Main implementation modules . . . 53
D.3 Analysis notebooks . . . 54
D.4 Execution flow . . . 55
D.5 Configuration . . . 56
D.6 Verification performed during the project cleanup . . . 57
D.7 How to rebuild the thesis artifacts . . . 57
D.8 Final reproducibility note . . . 58

E Appendix E: Additional Statistical Tests **59**
E.1 Mincer-Zarnowitz Forecast Efficiency Test 59
E.2 Residual Diagnostics: Ljung-Box and ARCH LM Tests 60
E.3 Harvey-Leybourne-Newbold Corrected Diebold-Mariano Test 61
E.4 Automated Validation Suite 62

F Appendix F: Market-Event Case Studies **64**

Објављени радови 72

List of Figures

1 Rolling dependency between Bitcoin and each of the five conventional assets plus Ethereum (six pairs in total) across all four window lengths (14, 30, 60, 90 days). The Fisher-transformed series exhibits the strong time-variation and persistence that motivate the forecasting framework. 19
2 Observed and predicted Fisher-transformed dependency for the Bitcoin–S&P 500 pair at the 30-day rolling window. Forecast series are generated under strict walk-forward evaluation with no use of future information. . . 27
3 Diebold–Mariano test results comparing Ridge (best overall RMSE across all models) against DCC-GARCH across all asset-pair and window-length combinations. Darker cells indicate stronger and more statistically significant evidence of Ridge superiority. 29
4 Rolling out-of-sample RMSE over time for the Bitcoin–S&P 500 dependency experiment at the 30-day correlation window ($w = 30$); the error is smoothed over a trailing 60-day window. The machine-learning model shown is Ridge, which has the lowest average RMSE overall (Table 3), compared against the DCC-GARCH benchmark. Elevated error episodes correspond to periods of abrupt regime transition in the dependency process. 30
5 Investor stress-warning signal for the Bitcoin–S&P 500 pair at the 30-day window. The upper panel displays the predicted stress probability from the logistic classifier together with the decision threshold; the lower panel marks flagged and missed stress events. 32
6 Normalized prices of the assets included in the study. 41
7 Rolling volatility across the asset universe. 42
8 Static correlation heatmap of the return series. 42
9 Observation coverage window per ticker. All assets share an identical aligned date range after the ETH-USD availability cutoff in November 2017. 43
10 Log-return series with extreme observations ($|z| > 5\sigma$) highlighted in red. Crypto assets exhibit sporadic but large outliers; conventional assets are more contained but not outlier-free. 44
11 Distribution of the 30-day rolling correlation before and after Fisher-z transformation for the Bitcoin–S&P 500 pair. The transformation improves normality and removes boundary compression. 45
12 RMSE by model and window for the representative Bitcoin–S&P 500 pair. 46
13 Model R^2 comparison for the 30-day window. 46
14 Improvement relative to the naive baseline. 46
15 Grid-search cross-validation RMSE for the representative Bitcoin–S&P 500 forecasting task. 47
16 Regularization path for the Ridge-based forecasting specification. 47
17 Random Forest feature importances for the Bitcoin–S&P 500 pair at the 30-day window. 47
18 Window sensitivity of the XGBoost-based forecasting layer. 48
19 Forecast scatter diagnostic for the Bitcoin–S&P 500 dependency experiment. 48
20 Forecast error distribution for the representative Bitcoin–S&P 500 dependency experiment. 48
21 Sensitivity analysis for the Bitcoin–NASDAQ pair. 49
22 Sensitivity analysis for the Bitcoin–gold pair. 49

23 Sensitivity analysis for the Bitcoin–silver pair. 49
24 Sensitivity analysis for the Bitcoin–UUP pair. 50
25 Sensitivity analysis for the Bitcoin–Ethereum pair. 50
26 Signal-layer output for the Bitcoin–NASDAQ relation at the 30-day window. The upper panel shows the predicted stress probability and the decision threshold; the lower panel marks flagged and missed stress events. . . 51
27 Master timeline 2018–2026: Bitcoin price, conventional-asset prices, and the 30-day rolling BTC–S&P 500 correlation, with all eleven landmark events marked. Red dotted lines = crash/shock events; green = rallies and all-time highs; blue = structural divergences; purple = institutional milestones. 64
28 Correlation regime map. Left: post-event BTC correlation (30-day average) with each conventional asset. Right: change in correlation relative to the 30-day pre-event average. Red/blue = strong positive/negative shifts. . 65
29 **Event 1 — COVID Black Thursday (12 March 2020).** Bitcoin fell about 37% in a single session, sharply outpacing the S&P 500's 10% drop. The 30-day BTC–S&P 500 correlation rose from near zero to about 0.52 — a liquidity-driven sell-off in which Bitcoin's diversification benefit temporarily disappeared. Source: Reuters 66
30 **Event 2 — FTX Collapse (8–11 November 2022).** SBF's exchange filed for bankruptcy; Bitcoin fell about 25% over the following two weeks. Paradoxically the S&P 500 *rose* that same week on a positive CPI print — producing a sharp short-term decoupling: the 14-day BTC–equity correlation fell from about 0.6 toward zero, though the 30-day measure barely moved (about 0.55). Source: BBC News 66
31 **Event 3 — SVB Banking Crisis (10 March 2023).** Silicon Valley Bank's failure triggered a Bitcoin rally of about 40% over the following week as investors sought alternatives to the banking system. Both Bitcoin and Gold rose (Gold about 8%) while equities fell — one of the rare episodes in the sample where the short-window correlation turned negative: the 14-day BTC–S&P 500 measure dipped to about -0.4, while the 30-day measure fell from 0.37 to 0.16. Source: The New York Times 67
32 **Event 4 — "Liberation Day" Tariff Shock (2 April 2025).** Sweeping tariffs on 185 countries caused the S&P 500 to fall about 12% in four trading days — its fastest drop since March 2020. Bitcoin fell alongside equities over the same week, while Gold climbed to record highs above \$3 000/oz. Although both risk assets sold off together — confirming that Bitcoin behaves as a risk asset, not a safe haven, during acute macro shocks — the 30-day rolling BTC–S&P 500 correlation showed no clear spike: it edged only from about 0.33 to 0.43, as the sharp partial recovery in the days that followed desynchronised the daily moves. The episode illustrates that even a major macro shock need not durably raise the rolling dependency. Source: Reuters 68

List of Tables

1 Asset universe used in the thesis. 17
2 Main groups of predictors used in the dependency-forecasting layer. 20
3 Average out-of-sample forecasting performance across all dependency experiments. 26
4 Average investor-signal performance by classifier family, aggregated across all experiments. 31
5 Average next-day return of the conventional asset on days flagged by the logistic signal versus days left clear, by asset pair (Logit, averaged over the four windows). Returns are in basis points per day; a positive difference means flagged days are, on average, worse. The flag rate is the share of days on which the signal is active. 33
6 Data quality summary per ticker 43
7 Balanced accuracy of the Logit signal layer by asset pair and rolling-window length. Family-level averages across all pairs and windows are reported in Table 4; this breakdown shows where the signal is informative. 52
8 Mincer-Zarnowitz forecast efficiency: percentage of 24 experiments passing the joint F-test ($p \geq 0.05$) and average coefficient estimates. 59
9 Mincer-Zarnowitz test: BTC-USD vs S&P 500, $w = 30$. F-statistic tests H_0: $\alpha = 0, \beta = 1$; p_{MZ} is the associated p-value. 60
10 Residual diagnostics: percentage of 24 experiments passing the Ljung-Box autocorrelation test and the ARCH LM heteroscedasticity test at $\alpha = 0.05$. 61
11 HLN-corrected DM test: ML models vs DCC-GARCH benchmark, BTC-USD vs S&P 500, $w = 30$. DM: original statistic (normal); $\mathrm{DM}_{\mathrm{HLN}}$: corrected statistic ($t_{n-1}$). All models beat DCC-GARCH at $p < 0.001$ under both tests. 62
12 Automated validation suite, summarised by test class. All 81 tests pass on the present outputs, with none skipped. The complete per-test log, including parametrised cases and per-test timings, is written to `outputs/results/test_results.xml` in the repository. 63

List of Abbreviations

Abbreviation	Full form
AR1	First-order autoregressive model
AUC	Area under the ROC curve
BTC	Bitcoin (`BTC-USD`)
CSV	Comma-separated values
DCC	Dynamic Conditional Correlation
DCC-GARCH	Dynamic Conditional Correlation GARCH(1,1)
DM	Diebold–Mariano (test)
EDA	Exploratory Data Analysis
ElasticNet	Elastic Net regularized regression
ETF	Exchange-Traded Fund
ETH	Ethereum (`ETH-USD`)
F1	F1 score (harmonic mean of precision and recall)
GARCH	Generalized Autoregressive Conditional Heteroskedasticity
GBM	Gradient Boosting Machine
GLD	SPDR Gold Shares ETF (`GLD`)
GPU	Graphics Processing Unit
GSPC	S&P 500 index (`^GSPC`)
HAR	Heterogeneous Autoregressive (model)
IXIC	NASDAQ Composite index (`^IXIC`)
MAE	Mean Absolute Error
ML	Machine Learning
NASDAQ	National Association of Securities Dealers Automated Quotations
OOS	Out-of-sample
RF	Random Forest
RMSE	Root Mean Squared Error
ROC	Receiver Operating Characteristic
S&P 500	Standard & Poor's 500 index
SLV	iShares Silver Trust ETF (`SLV`)
UUP	Invesco DB US Dollar Index Bullish Fund (`UUP`)
XGB	XGBoost (Extreme Gradient Boosting)

1 Introduction

1.1 Motivation and research context

Over the last decade, Bitcoin has developed from a niche technological experiment into an asset class held and monitored by institutional investors. This shift has a direct consequence for risk management: once a previously isolated market becomes large enough to attract substantial capital, its price movements begin to affect portfolios that hold no cryptocurrency directly. The central concern for risk management is therefore the time variation in this influence — *how* and *when* the dependence between Bitcoin and conventional markets changes.

This thesis focuses on *time-varying intermarket dependency*: the evolving statistical relationship between Bitcoin and conventional assets such as equity indices, gold, silver, and the dollar. The focus is on whether this correlation is predictable at all, and whether cryptocurrency price dynamics contain an early-warning signal for stress in traditional markets, rather than on its average level.

The practical motivation is concrete. A risk manager does not require an accurate point forecast of returns to benefit from this kind of analysis. What matters is the ability to recognise when co-movement patterns are changing — when a market regime is shifting from decorrelated to tightly coupled, or when the probability of a stressful day in conventional markets is rising. Even a modest and statistically reliable signal of this type can be useful, provided its limitations are understood.

1.2 Research questions

The thesis is structured around two connected layers. The first is a forecasting problem: can rolling correlation between Bitcoin and a conventional asset be predicted one step ahead, and how do machine-learning models compare to a classical econometric benchmark under strict out-of-sample evaluation? The second is an applied question: can those forecasts be turned into a practical warning signal for investors?

More specifically, the four research questions addressed are:

1. Is the rolling dependency between Bitcoin and conventional assets forecastable out of sample?
2. Do machine-learning models outperform a leakage-safe DCC-GARCH benchmark?
3. Do machine-learning models add value beyond simple persistence-based baselines such as AR1?
4. Can crypto-derived dependency forecasts support a practically useful stress-warning signal for conventional markets?

1.3 Contributions of the thesis

To address these questions, a reproducible Python pipeline was built from scratch. It downloads price data, constructs log-return series, computes rolling correlations across multiple window lengths, engineers lagged and volatility-based features, and evaluates a range of models within a walk-forward experimental design. The model set includes a

naive random-walk baseline and an AR(1) autoregressive model, a Heterogeneous AutoRegressive (HAR) model using daily, weekly, and monthly dependency lags, regularised linear models (Ridge, ElasticNet), an adaptive inverse-RMSE-weighted ensemble, tree-based ensembles (Random Forest, GBM, XGBoost), and a leakage-safe DCC-GARCH(1,1) benchmark. On top of the forecasting stage, a second layer converts predictions into a binary stress classifier for conventional assets.

The main empirical finding is that the dependency process is forecastable out of sample, but that most of this predictability derives from the target's own past rather than from new cross-asset information. Ridge, AR(1), and HAR form a near-indistinguishable leading cluster: three structurally different models that reach essentially the same accuracy because each relies on the strong serial persistence of the rolling correlation. This convergence is itself an informative result, since it indicates that the target has a nearly linear autoregressive structure and that additional model complexity provides little benefit. Machine learning nonetheless adds clear value relative to DCC-GARCH, and it is most useful in the stress-classification task, where regularised linear classifiers outperform tree-based ensembles.

Three concrete contributions follow from this work. First, a reproducible Python pipeline makes the full empirical record replicable from raw data to final figures. Second, a systematic comparison of ten models, ranging from naive persistence to XGBoost and DCC-GARCH under a consistent walk-forward design, quantifies where machine learning adds value and where it does not. Third, an explicit mapping from rolling-correlation forecasts to a binary stress classifier translates the statistical findings into a form usable by a risk practitioner.

1.4 Thesis outline

Section 2 reviews the relevant theory and literature: time-varying dependence, financial contagion, machine-learning forecasting methods, and the Diebold–Mariano test. Section 3 describes the data, target construction, feature engineering, model specifications, and evaluation design. Section 4 reports empirical results for both layers. Section 5 interprets and qualifies those results. Section 6 concludes.

2 Theoretical Background and Literature Review

2.1 Why intermarket dependency matters

The way assets move together is central to financial theory, since diversification, contagion, hedging, and portfolio risk all depend on it. The textbook mean-variance framework treats this co-movement as a single constant correlation, but decades of evidence indicate that the static picture does not hold. Empirical correlations drift over time, react more strongly to negative news than to positive news, and rise sharply during market downturns, precisely when diversification would be most valuable [1]. Forecasting *time-varying dependence* is therefore of direct practical importance for risk management.

These considerations are even more pronounced when one side of the pair is a cryptocurrency [2]. Bitcoin trades continuously on a globally accessible market, responds quickly to changes in sentiment and funding liquidity, and is interpreted through several competing narratives: as an inflation hedge, a speculative asset, an indicator of risk appetite, and an instrument sensitive to regulation. Because so many factors influence it, Bitcoin does not occupy a single stable role within a portfolio. Depending on the stage of the business and credit cycle, it can behave like a high-beta growth stock, a liquidity-sensitive risk proxy, or an idiosyncratic sentiment indicator. This instability is what makes its relationship with conventional assets a natural candidate for a time-varying treatment.

2.2 Cryptocurrencies and conventional assets

Research at the intersection of cryptocurrency and conventional markets has grown rapidly over the past decade and can be grouped into several broad lines. The first studies Bitcoin in isolation, characterising its return distribution, volatility clustering, tail risk, and weak-form efficiency. The second addresses the portfolio question of whether Bitcoin provides effective diversification against equities, gold, oil, or major currencies. The third line, which is closest to this thesis, studies connectedness, contagion, and the spillover of returns and volatility across markets, with particular attention to crises and stress episodes.

A recurring result across this literature is that the relationship is unstable [3, 4]. In calm markets, Bitcoin's association with broad equity indices is weak or inconsistent in direction; in pronounced risk-on or risk-off episodes, it tightens sharply [5, 6]. Precious metals co-move with Bitcoin more weakly and less reliably [7, 8], consistent with their different investor base and long-standing safe-haven role. Currencies provide a further dimension, serving as a proxy for global funding liquidity [4].

These patterns motivate modelling dependence as a time-varying quantity rather than a fixed number. A single full-sample correlation would average out the behaviour of interest. Treating dependence as a time-varying process instead accommodates slow regime drift, sudden breaks, and the fact that different asset pairs and window lengths change at different speeds and in different directions.

2.3 Rolling correlation as a forecasting target

The rolling window correlation estimator is conceptually transparent and computationally tractable. For two return series $\{r_{1,t}\}$ and $\{r_{2,t}\}$, the rolling correlation at time t over a

window of length w is defined as

$$\hat{\rho}_t^{(w)} = \frac{\sum_{s=t-w+1}^{t}(r_{1,s} - \bar{r}_{1,t})(r_{2,s} - \bar{r}_{2,t})}{\sqrt{\sum_{s=t-w+1}^{t}(r_{1,s} - \bar{r}_{1,t})^2} \cdot \sqrt{\sum_{s=t-w+1}^{t}(r_{2,s} - \bar{r}_{2,t})^2}}, \tag{1}$$

where $\bar{r}_{k,t}$ denotes the in-window mean of series k. Traversing this estimator across all t yields a time series $\{\hat{\rho}_t^{(w)}\}$ that characterizes the temporal evolution of intermarket co-movement.

As a forecasting target, rolling correlation has several advantages. It is easy to interpret, since positive values indicate that the two assets move together and negative values indicate a hedging relationship. It is comparable across asset pairs and window lengths, which makes a systematic multi-asset study feasible. It also keeps the analysis focused on how market structure is changing rather than on the level of any single return series.

Its main complication is strong serial persistence. Because $\hat{\rho}_t^{(w)}$ is constructed from overlapping windows, consecutive estimates share $w - 1$ of their observations, so the series is heavily autocorrelated by construction. This persistence is likely to dominate any forecast and may leave complex models with little to add beyond a simple autoregression. The thesis treats this persistence as a substantive empirical property of the dependency process, and as a central object of analysis in its own right.

To improve the statistical behavior of the target, the thesis applies the Fisher z-transformation:

$$z_t^{(w)} = \frac{1}{2} \ln\left(\frac{1 + \hat{\rho}_t^{(w)}}{1 - \hat{\rho}_t^{(w)}}\right), \tag{2}$$

which stretches the bounded values $\hat{\rho} \in (-1, 1)$ onto the whole real line and gives them a sampling distribution much closer to Gaussian. It is standard practice for correlation-like quantities and tames the variance that otherwise blows up near ± 1.

The main alternative is a copula-based approach [9], which models the marginals and the dependence structure separately and therefore captures tail dependence and nonlinear co-movement that Pearson correlation cannot represent. This thesis retains rolling Pearson correlation for four reasons: (i) it is directly interpretable for risk managers; (ii) it is cheap enough to recompute at every step of the walk-forward loop; (iii) it lines up with the DCC-GARCH benchmark, whose output is exactly the conditional correlation matrix R_t [10]; and (iv) the Fisher z-transformation already handles the boundary behaviour. The Pearson restriction is a genuine limitation, and copula extensions are the natural next step.

2.4 Econometric benchmark: DCC-GARCH

The Dynamic Conditional Correlation GARCH model, introduced by Engle [11], provides the classical econometric benchmark for the present study. The DCC-GARCH framework decomposes the conditional covariance matrix H_t of a vector of returns $\mathbf{r}_t$ according to

$$H_t = D_t R_t D_t, \tag{3}$$

where $D_t = \mathrm{diag}(h_{1,t}^{1/2}, h_{2,t}^{1/2})$ is a diagonal matrix of conditional standard deviations and R_t is the time-varying conditional correlation matrix. Each conditional variance $h_{k,t}$ follows

a univariate GARCH(1,1) process,

$$h_{k,t} = \omega_k + \alpha_k \varepsilon_{k,t-1}^2 + \beta_k h_{k,t-1}, \tag{4}$$

where $\varepsilon_{k,t} = r_{k,t}/h_{k,t}^{1/2}$ is the standardized residual. The standardized residuals $\tilde{\varepsilon}_t = D_t^{-1} r_t$ are then used to update the pseudo-correlation matrix Q_t via

$$Q_t = (1 - a - b)\bar{Q} + a\, \tilde{\varepsilon}_{t-1} \tilde{\varepsilon}_{t-1}^\top + b\, Q_{t-1}, \tag{5}$$

where $\bar{Q}$ is the unconditional covariance of standardized residuals, and the scalar parameters $a \geq 0$, $b \geq 0$, $a + b < 1$ govern the speed of mean reversion and the persistence of conditional correlation. The conditional correlation matrix is recovered as

$$R_t = \operatorname{diag}(Q_t)^{-1/2}\, Q_t\, \operatorname{diag}(Q_t)^{-1/2}. \tag{6}$$

DCC-GARCH is a natural benchmark in this setting, since it models time-varying comovement under the heteroskedastic conditions that both crypto and equity returns display. Its weakness follows from the same parametric structure: a fixed functional form has limited capacity to adapt to nonlinearities or abrupt breaks. A further, more subtle issue is information leakage: fitting the model on the full sample before reading off "out-of-sample" predictions implicitly uses future information. To avoid this, the thesis re-estimates DCC-GARCH on an expanding window at every evaluation step, in the same way as for the machine-learning models.

2.5 Machine-learning approaches to forecasting

Machine-learning methods are included here for their ability to capture nonlinearities, threshold effects, and interactions that are difficult to specify in a parametric model [12]. They are not assumed to outperform in every case; the thesis evaluates them against both the DCC-GARCH benchmark and simple persistence baselines, with two aims: to identify where their strengths lie, and to determine the conditions under which they add predictive value.

2.5.1 The HAR model and persistence-based forecasting

The Heterogeneous Autoregressive (HAR) model, introduced by Corsi [13], provides a natural benchmark for forecasting persistent financial time series. The HAR model approximates long-memory dynamics by summing three autoregressive components operating at daily, weekly, and monthly horizons:

$$z_t = \phi_0 + \phi_d z_{t-1} + \phi_w \bar{z}_{t-5:t-1} + \phi_m \bar{z}_{t-22:t-1} + \varepsilon_t, \tag{7}$$

where $\bar{z}_{t-k:t-1}$ is the arithmetic mean of z over the preceding k periods. The model is particularly well suited to rolling-correlation targets, which already carry a moving-average smoothing from their overlapping windows, the structure HAR is designed to exploit [14]. In this implementation, HAR is a restricted OLS on three regressors, the latest value (daily), the 5-day mean (weekly), and the 22-day mean (monthly), evaluated under the same walk-forward protocol as the other models.

2.5.2 Regularized linear models

Regularized linear models, including Ridge regression and Elastic Net, represent strong baselines in structured tabular forecasting settings. Ridge regression imposes an ℓ_2 penalty on the coefficient vector, shrinking estimates toward zero to reduce estimation variance without introducing sparsity. Elastic Net combines ℓ_1 and ℓ_2 regularization:

$$\hat{\boldsymbol{\beta}} = \arg\min_{\beta} \left\{ \frac{1}{n} \sum_{i=1}^{n} (y_i - \mathbf{x}_i^\top \boldsymbol{\beta})^2 + \lambda \left[(1-\alpha) \|\boldsymbol{\beta}\|_2^2 + \alpha \|\boldsymbol{\beta}\|_1 \right] \right\}, \tag{8}$$

where $\lambda > 0$ sets the overall strength of the penalty and $\alpha \in [0, 1]$ splits it between the two terms [15]. The ℓ_1 component drives some coefficients to zero, performing implicit variable selection, which is useful when many lagged predictors are weak or redundant. This is well suited to the feature set used here, which contains many overlapping transformations of correlated return and volatility series.

Despite their linearity, these models often perform competitively on smooth, persistence-driven problems. When the signal-to-noise ratio is low and the most informative feature is a lag of the target, the bias introduced by the linearity assumption is typically outweighed by the reduction in variance that regularization provides.

2.5.3 Tree-based ensemble models

Random Forests construct an ensemble of decision trees, each trained on a bootstrapped subsample and a random subset of features, and aggregate their predictions to reduce variance and improve generalization [16]. Gradient Boosting Machines instead build trees sequentially, with each new tree fitted to the pseudo-residuals of the current ensemble [17]. XGBoost extends this boosting framework with second-order gradient approximations, ℓ_1 and ℓ_2 regularization terms on tree structure, and computational optimizations that facilitate large-scale application [18].

Tree ensembles are natural candidates because the relationship between crypto features and intermarket dependence may be nonlinear and rich in interactions. The information that Bitcoin's realized volatility carries about future equity dependence, for example, may depend on whether the correlation is already high. Tree-based methods can detect and exploit such conditional patterns automatically, without the interactions being specified in advance.

None of this guarantees that tree-based methods will be effective for one-step-ahead forecasts of a highly persistent target. When the target is governed almost entirely by its most recent value, the additional flexibility may contribute little predictive signal while incurring a substantial variance penalty. This trade-off is one of the main reasons the thesis relies on a disciplined walk-forward comparison rather than a single summary statistic.

2.6 Walk-forward evaluation and information leakage

Temporal ordering is fundamental in financial forecasting and constrains how the evaluation must be designed. A random train-test split is inappropriate, because it disrupts the causal ordering and allows a model trained on period $t + k$ to "predict" period t. A valid design is one in which every forecast uses only information available at or before its own origin.

The thesis therefore adopts a walk-forward procedure: train on an expanding history, produce a one-step-ahead prediction, extend the window by one day, and repeat. To keep the computational cost manageable, models are refitted at regular intervals rather than at every step, a compromise that also reflects how a forecasting system would operate in practice. In either case, no future information can enter the forecast.

The evaluation protocol is as important as the choice of model. A complex model with an undetected leak can appear to outperform a simpler, correctly evaluated competitor, which would invalidate the comparison. Because the objective is to reach robust conclusions, a leakage-free evaluation is treated as a strict requirement.

2.7 Model comparison and the Diebold–Mariano test

Ranking models by their average out-of-sample error is informative but not conclusive: it says nothing about whether a gap is real or just sampling noise. The Diebold–Mariano test closes that gap, formally testing the null that two forecasts are equally accurate [19].

Let $e_{1,t}$ and $e_{2,t}$ denote the forecast errors of models 1 and 2 at time t, and let $d_t = g(e_{1,t}) - g(e_{2,t})$ denote the loss differential under a loss function $g(\cdot)$ (typically squared or absolute error). The DM test statistic is

$$\mathrm{DM} = \frac{\bar{d}}{\sqrt{\hat{V}(\bar{d})\,/\,T}} \xrightarrow{d} \mathcal{N}(0,1), \tag{9}$$

where $\bar{d} = T^{-1}\sum_{t=1}^{T} d_t$ is the mean loss differential, $\hat{V}(\bar{d})$ is a heteroskedasticity- and autocorrelation-consistent (HAC) variance estimator, and T is the number of evaluation periods. Under the null of equal predictive accuracy, the statistic is asymptotically standard normal.

The DM test serves two purposes here. First, it assesses whether the best machine-learning configuration genuinely outperforms the leakage-safe DCC-GARCH benchmark. Second, it assesses whether any improvement over a plain autoregression or naive persistence baseline survives once sampling noise is taken into account. The second comparison is the more informative: if a model cannot significantly outperform AR(1), then most of the forecastability resides in the target's own persistence rather than in information extracted from the cross-asset features.

2.8 From dependency forecasting to investor signaling

Accurate forecasting of rolling correlation would in itself constitute a complete academic exercise. Because the thesis also has a practical motivation, however, it adds a further step: converting the forecasts into a form on which an investor can act, framed around the risk-management decision itself.

This step is implemented as a second-stage signal layer. Rather than predicting the direction of tomorrow's return, it estimates the probability that the following day falls in a *stress regime*: a day of elevated downside risk in the conventional asset under study. This formulation is closer to the way risk management operates in practice, where the objective is to flag deteriorating conditions early enough to respond rather than to predict every small movement.

Keeping the two stages separate makes the design more transparent. Lower forecast error and genuine usefulness for portfolio decisions are distinct properties, and conflating them can overstate a result. The two-layer structure allows the thesis to report on each dimension separately.

2.9 Positioning of the present thesis

The thesis lies at the intersection of financial econometrics, machine learning, and applied risk analysis. It does not claim that cryptocurrency data solve the market-timing problem. The claim is narrower: Bitcoin's price dynamics carry statistically detectable information about how intermarket dependence evolves, and this information can support a modest early-warning signal for conventional markets.

Its main contribution is to establish that dynamic intermarket dependence is forecastable, and to compare persistence baselines, machine-learning models, and DCC-GARCH on an equal walk-forward footing. The investor signal layer extends the work in a practical direction without claiming more than the evidence supports.

3 Methodology

3.1 Research design

The design has two layers. The first is a regression problem: forecast the next value of the (transformed) rolling correlation between Bitcoin and another asset. The second is a classification problem: take that forecast, add a few crypto-derived features, and estimate the probability that the conventional asset has a stress day. The split mirrors the difference between measuring something and acting on it.

The design is deliberately conservative. Every step that could leak future information — target construction, feature alignment, benchmark estimation, and signal generation — is performed inside the walk-forward loop rather than tuned on in-sample fit. The single exception is hyperparameter selection, which is carried out once on a representative experiment rather than re-tuned at every step; as shown in Chapter 5, the results are essentially flat in the regularisation strength, so this simplification does not materially affect the conclusions. The aim throughout is to preserve correct temporal ordering and prevent accidental look-ahead.

3.2 Asset universe and data source

The empirical analysis uses daily market data obtained through the `yfinance` interface [20]. Bitcoin, represented by `BTC-USD`, serves as the base cryptocurrency throughout the study. The comparison set includes the following assets:

Table 1: Asset universe used in the thesis.

Ticker	Interpretation
`BTC-USD`	Bitcoin, base cryptocurrency
`ETH-USD`	Ethereum, crypto reference asset
`^GSPC`	S&P 500 index
`^IXIC`	NASDAQ Composite index
`GLD`	Gold ETF
`SLV`	Silver ETF
`UUP`	U.S. Dollar Index ETF proxy

The choice of assets is substantive, not arbitrary. The two equity indices stand in for broad U.S. risk appetite. The gold and silver ETFs bring in precious-metal exposure, with its mix of inflation, safe-haven, and commodity narratives. The dollar-index proxy captures global dollar strength and defensive positioning. Ethereum is there so the study can contrast crypto-to-crypto dependence with crypto-to-conventional dependence.

The sample runs from 9 November 2017 to 16 May 2026, giving 3,053 daily observations per asset.[1] The start date is set by Ethereum: Yahoo Finance has no ETH-USD prices before then, so earlier dates drop out of the aligned panel. What remains is balanced: all seven tickers share one date index, with no gaps left after alignment and forward-filling of non-trading days up to two consecutive calendar days.

[1]The pipeline downloads data to the most recent available trading day; the figures reported here correspond to the final run of the pipeline at the time of writing.

Normalized price paths, thirty-day rolling-volatility profiles, and the full-sample return correlation matrix for the asset universe are collected in Appendix A (Figures 6, 7, and 8). The much higher, more episodic volatility of the crypto assets and the low static Bitcoin–conventional correlations they display are what motivate treating cross-asset dependency dynamically rather than as a single full-sample number.

3.3 Return construction and preprocessing

Every price series is turned into daily log returns. Log returns are the natural choice here: they add up over time and are the standard currency of both econometric and machine-learning work on financial series. For a price series P_t, the return is

$$r_t = \log P_t - \log P_{t-1}.$$

Missing observations are dealt with by aligning on common dates; nothing downstream (no target, no feature) is built until the return matrix is synchronized.

Preprocessing is cache-aware. If the raw prices are already on disk, they are reused, which keeps runs reproducible; if the price structure or the ticker set changes, the processed returns are rebuilt from scratch. That way nothing stale lingers, and nothing is recomputed needlessly.

3.4 Target construction: rolling dependency

For each pair consisting of Bitcoin and another asset, a rolling correlation series is computed over windows of 14, 30, 60, and 90 calendar days. Let $r_t^{(b)}$ denote the return of Bitcoin and $r_t^{(a)}$ the return of the other asset. For a given window length w, the dependency target at time t is

$$\rho_t^{(w)} = \operatorname{corr}\big(r_{t-w+1:t}^{(b)}, r_{t-w+1:t}^{(a)}\big).$$

The resulting series is then transformed using the Fisher transformation

$$z_t^{(w)} = \frac{1}{2}\log\left(\frac{1+\rho_t^{(w)}}{1-\rho_t^{(w)}}\right) = \operatorname{arctanh}\big(\rho_t^{(w)}\big),$$

which maps the bounded correlation domain $(-1, 1)$ into an unbounded real-valued space. In the empirical implementation, the one-step-ahead value of the transformed dependency series is used as the main forecasting target. The implementation clips the raw correlation away from the boundary to prevent numerical overflow:

Listing 1: Fisher z-transform and target construction (`pipeline.py`).

```
def fisher_z(r: pd.Series, eps: float = 1e-6) -> pd.Series:
    # Clip away from +/-1 to prevent arctanh overflow
    return np.arctanh(r.clip(-1 + eps, 1 - eps))

def build_target(returns, base, other, window, horizon):
    corr = returns[base].rolling(window).corr(returns[other]).dropna()
    # Shift by horizon to get the one-step-ahead target
    target = corr.shift(-horizon).dropna()
    return fisher_z(target)
```

One consequence of all this is that the target is smooth to begin with, and far more persistent than a raw return series. That persistence is one of the central empirical facts of the thesis, and it shows up everywhere in the results.

The rolling dependency series for all six pairs across the four window lengths is shown in Figure 1. Its pronounced time-variation and persistence are the properties the forecasting framework is designed to exploit.

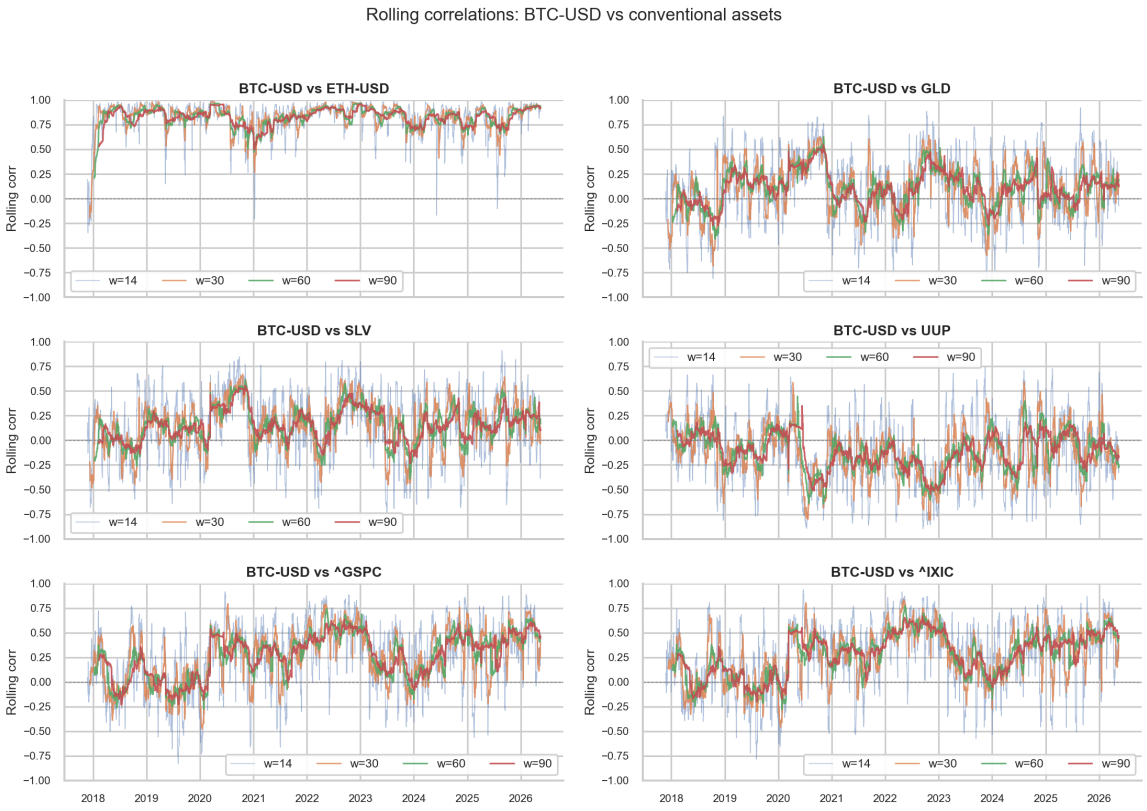


Figure 1: Rolling dependency between Bitcoin and each of the five conventional assets plus Ethereum (six pairs in total) across all four window lengths (14, 30, 60, 90 days). The Fisher-transformed series exhibits the strong time-variation and persistence that motivate the forecasting framework.

3.5 Feature engineering

The features are built around quantities a financial reader can interpret. The aim is not a sprawling black-box design matrix but a compact set of variables that could plausibly move future dependency.

The features fall into several groups:

Table 2: Main groups of predictors used in the dependency-forecasting layer.

Group	Description
Dependency lags	Recent values of the rolling dependency target
Dependency momentum & regime	Trend (5- and 20-day change) and trailing z-score of the target
Return lags	Lagged returns of Bitcoin and the paired asset
Rolling means	Local average return over short windows
Rolling volatility	Local standard deviation of returns
Spread features	Absolute and signed gaps between crypto and asset behavior
Correlation regime gap	Short-window minus long-window correlation

The dependency lags carry most of the predictive weight, because the target is persistent by construction. Dependency momentum (`dep_mom`) and the trailing z-score (`dep_zscore`) summarise the recent trend and the standardised level of the correlation. Return lags allow the models to respond to recent price direction, while the rolling-mean, volatility, and dispersion features describe the local risk regime. The regime gap (`corr_diff`) measures how far short-term dependency has moved from its longer-term baseline, which can act as an early indicator of a regime shift. Listing 2 shows how the feature matrix is assembled.

Listing 2: Core feature engineering routine (`pipeline.py`; abbreviated — the full routine additionally builds rolling-mean, dispersion, volatility-ratio, and lagged squared-return features).

```
# Dependency lags (extended: up to quarterly)
for lag in [1, 2, 5, 10, 20, 60]:
    features[f"dep_lag{lag}"] = y_current.shift(lag)

# Dependency momentum (trend of the correlation series)
features["dep_mom_5"]  = y_current.diff(5)
features["dep_mom_20"] = y_current.diff(20)

# Dependency regime: z-score vs trailing 252-day history
features["dep_zscore"] = (y_current - dep_roll_mean) / (dep_roll_std + 1e
    ↪ -8)

# Volatility of both assets, at the pair window and at 60 days
features["vol_base"]     = returns[base].rolling(window).std()
features["vol_other"]    = returns[other].rolling(window).std()
features["vol_base_60"]  = returns[base].rolling(60).std()
features["vol_other_60"] = returns[other].rolling(60).std()

# Return lags for both assets
for lag in [1, 2, 5, 10, 20]:
    features[f"r_base_lag{lag}"]  = returns[base].shift(lag)
    features[f"r_other_lag{lag}"] = returns[other].shift(lag)

# Correlation regime gap: short-window minus long-window correlation
corr_short = rolling_corr(returns[base], returns[other], max(5, window //
    ↪ 4))
```

```
corr_long  = rolling_corr(returns[base], returns[other], window)
features["corr_diff"] = corr_short - corr_long
```

The signal layer takes the best available dependency forecast for the pair and combines it with a handful of crypto-derived predictors into a smaller, classification-oriented feature set. In effect, the second layer sees both the current state of the crypto market and where the dependency process is heading.

3.6 Forecasting models

3.6.1 Baseline and autoregressive models

Three persistence-based models anchor the set. `Naive_Last` just carries the last observed value forward, with no fitting at all. `AR1` fits a single autoregressive lag by OLS. `HAR` (Heterogeneous AutoRegressive, after Corsi [13]) adds two more horizons: the latest value, its five-day average (weekly), and its twenty-two-day average (monthly), fitted as a restricted OLS on the expanding window at each refit. These baselines are not a formality: because the target is persistent by construction, any more complex model must demonstrate a clear improvement over them to justify its use.

3.6.2 Regularized linear models

The linear block is Ridge and ElasticNet, each fitted after standardization (regularization is meaningless if the features sit on different scales). One notational warning: the penalty strength below is scikit-learn's `alpha`, which is the λ of Eq. (8), not the ℓ_1/ℓ_2 mixing ratio α of that same equation (in the code that ratio is `l1_ratio`). Ridge (ℓ_2, `alpha` $= 0.5$) should do well when many predictors each carry a little signal. ElasticNet (`alpha` $= 0.005$, with an even ℓ_1/ℓ_2 split) is more aggressive and can zero out redundant lags outright.

3.6.3 Tree ensembles and adaptive ensemble

The non-linear block is Random Forest, `HistGradientBoostingRegressor` (a LightGBM-style histogram method, roughly ten times faster than the old gradient-boosting code), and XGBoost, which runs on the GPU when it can and quietly falls back to CPU when it cannot. On top of these sits an `Ensemble`: at each step it forms an inverse-RMSE weighted average of all the non-naive models, with the weights recomputed from the last 60 out-of-sample errors — or, during the first 60 steps before that many errors exist, from whatever errors have accumulated so far.

The complete model set (ten specifications in total) is initialized as follows:

Listing 3: Model definitions used in the walk-forward experiment (`pipeline.py`; simplified — `Naive_Last` and `HAR` are `None` here and are produced directly inside the walk-forward loop rather than through `.fit`).

```
model_specs = {
    "Naive_Last": None,          # persistence baseline (no fit required)
    "AR1":        LinearRegression(),
    "HAR":        None,          # heterogeneous AR -- fitted via OLS on 3
    ↪ lags
```

```
                                # (dep_lag1, avg5, avg22); see walk-forward loop
    "Ridge":      make_pipeline(StandardScaler(), Ridge(alpha=0.5)),
    "ElasticNet": make_pipeline(
                      StandardScaler(),
                      ElasticNet(alpha=0.005, l1_ratio=0.5, max_iter=5000)
                  ),
    "RF":  RandomForestRegressor(
               n_estimators=200, max_depth=10,
               min_samples_leaf=5, n_jobs=-1
           ),
    "GBM": HistGradientBoostingRegressor(     # LightGBM-style histogram algorithm
               max_iter=300, max_depth=4,
               learning_rate=0.02, min_samples_leaf=5,
               l2_regularization=0.1
           ),
    "XGB_GPU": xgb.XGBRegressor(
               device="cuda", n_estimators=500,
               max_depth=6, learning_rate=0.02,
               subsample=0.8, colsample_bytree=0.8
           ),
}
# "Ensemble": adaptive inverse-RMSE weighted average of the ML models
#             (excludes Naive_Last and the DCC-GARCH benchmark).
# Weights are recomputed at each step from the last 60 out-of-sample errors .
```

3.6.4 DCC-GARCH benchmark

The DCC-GARCH benchmark is implemented in a separate module and is leakage-safe by construction. Rather than fitting DCC once on the full sample and then treating that fit as out-of-sample, the benchmark is re-estimated on the training window alone and rolled forward from there. This is computationally slower, but it places the benchmark on the same out-of-sample footing as the machine-learning models.

3.7 Investor signal layer

The signal layer reformulates the problem as early-warning classification. The label is a *stress day* for the conventional asset rather than next-day return direction. Day $t+1$ is labelled as stress if the asset's return falls below $-0.75\,\sigma_t$, where σ_t is the trailing 20-day standard deviation of returns computed from information available at time t. This threshold produces a positive (stress) class of roughly 15–18% of days, depending on the asset (15.2% for the S&P 500 up to 17.5% for the dollar index), and its sensitivity is examined over the range $[0.5,\, 2.0]\,\sigma$ in the robustness analysis. The threshold is chosen to capture moves that are worse than a typical down day rather than only extreme tail events, which matches the downside-protection use case of the signal.

Three classifiers compete here: logistic regression, a random-forest classifier, and a gradient-boosted one. Each emits a probability, which is compared against a threshold

to flag the next day as elevated-risk or not. The output is therefore a probability-guided warning, not an opaque buy/sell command.

3.8 Walk-forward estimation framework

Every experiment uses the same expanding-window walk-forward. Start with the first T_0 observations; at each origin $t \geq T_0$, fit on everything up to t and predict $t + 1$; continue to the end of the sample. Refitting happens periodically rather than at every step, which cuts the compute cost without ever breaking causal order.

Listing 4: Expanding-window walk-forward loop (`pipeline.py`).

```
for t in range(min_train, n_obs):
    train_idx = np.arange(0, t)

    # Refit all models on the expanding training window periodically
    if (t - last_refit) >= refit_every:
        last_refit = t
        for name, model in model_specs.items():
            if model is not None:
                model.fit(X_values[train_idx], y_values[train_idx])
                fitted[name] = model

    # One-step-ahead prediction - uses only information up to t
    for name in fitted:
        preds[name][t] = fitted[name].predict(X_values[[t]])[0]
```

This design has three benefits: it reproduces realistic forecasting conditions, it keeps the benchmark comparison fair, and it produces a complete time series of out-of-sample errors. The last point is important, since those errors feed not only the average metrics but also the rolling diagnostics and the Diebold–Mariano tests.

3.9 Evaluation metrics

The forecasting layer is evaluated with three metrics: mean absolute error (MAE), root mean squared error (RMSE), and the out-of-sample R^2. Rankings are based on RMSE, which penalises large errors more heavily and is well suited to a smooth, continuous target.

For the signal layer, four numbers matter: Balanced Accuracy, the F1 score on the downside class, the area under the ROC curve (AUC), and the exit rate. Balanced Accuracy is emphasised because stress days are rare and the classes are imbalanced. The F1 score on the downside class keeps the focus on detecting the adverse days. The exit rate, that is, how often the signal would move the investor out of the market, links statistical quality to how the signal would actually be used.

3.10 Model comparison methodology

When models are close, average metrics can mislead. So the thesis runs the Diebold–Mariano test on the loss series of selected pairs. The headline comparison is the best non-benchmark machine-learning model against DCC-GARCH; the secondary one pits that same model against the naive persistence baseline.

The setup is deliberately transparent. The thesis does not bury the possibility that the best model overall is a simple baseline; instead it uses the DM framework to separate three different levels of claim:

1. whether dependency is forecastable at all,
2. whether machine learning improves upon a classical econometric benchmark,
3. whether machine learning adds value beyond persistence.

A caveat applies to the second and third comparisons. When the simpler model is nested inside the richer one — as Naive_Last and AR1 are inside Ridge or the tree ensembles — the standard Diebold–Mariano statistic is not strictly valid, because its asymptotic normal distribution assumes non-nested forecasts. Tests built for nested models, such as Clark–West or Giacomini–White [21, 22], would be more rigorous, and the small-sample HLN correction reported in Appendix E mitigates over-rejection but not nesting. The comparisons against the persistence baselines therefore carry only indicative weight; the headline Ridge-versus-DCC-GARCH comparison, which is non-nested, is unaffected. A separate consideration concerns the variance estimator. The DM statistics use a Newey–West HAC estimator with zero lags, which is standard at the one-step horizon but assumes loss differentials that are not autocorrelated. Because the rolling-correlation target is built from overlapping windows, the loss differentials are autocorrelated by construction, so the zero-lag variance can be understated and the statistic correspondingly inflated. Re-estimating the variance with a lag of the order of the rolling-window length would give a more conservative statistic; given the very large margins against DCC-GARCH (DM ≈ 27), the significance of the headline comparison is unlikely to be overturned, but the precise magnitudes should be interpreted with this in mind.

The thesis organizes its empirical conclusions around exactly these three levels.

3.11 Implementation and reproducibility

The whole workflow is Python, organized as a modular codebase. A main pipeline ties the steps together: read the config, compute returns, build targets and features, fit models, write prediction files, save figures, export LaTeX tables. Separate modules handle the DCC walk-forward benchmark, notebook support, and the signal layer. This modular organisation supports the goal of making the thesis a reproducible artifact, not only a written argument.

Everything lands in a structured output tree: metrics, DM tests, and signal results as CSV; thesis-ready tables as LaTeX; diagnostics as figures. The point is to keep manual post-processing to a minimum and let the analysis feed the writing directly.

3.12 Methodological expectations

Before the results, it is worth saying what success would even look like. It does not mean finding a model that predicts every conventional market from Bitcoin; no serious forecasting study should promise that. A fairer bar has three parts: the dependency target should display out-of-sample forecastability, the benchmark comparison should reveal whether machine learning improves on DCC-GARCH, and the investor signal layer should provide non-random and practically interpretable stress warnings. These thresholds are

modest but defensible: low enough to be met without overstatement, yet high enough to be meaningful.

Three further principles guide how the results are read. First, no single metric is treated as the verdict: the regression layer always reports RMSE, MAE, and R^2 together, and the classification layer reports Balanced Accuracy, F1, AUC, and exit rate together. Second, performance gaps are judged significant by the Diebold–Mariano test, not by eyeballing averages. Third, the analysis refuses to cherry-pick a window after the fact; all four (14, 30, 60, 90 days) are reported side by side, so no finding can rest on a lucky choice of specification.

Behind all three is the same commitment to methodological transparency. In-sample performance can appear strong yet deteriorate substantially under proper out-of-sample testing. The walk-forward design, the leakage-safe benchmark, and the all-windows evaluation together guard against the kind of inflated claims to which this literature is prone.

4 Empirical Results

4.1 Overview of the empirical output

The pipeline leaves behind a full, internally consistent trail of evidence: out-of-sample metrics, model rankings, Diebold–Mariano tests, signal-layer classification statistics, the prediction series themselves, and diagnostic figures. That breadth is deliberate. The empirical case here does not hang on a single table; it rests on whether the same story holds up from several angles at once: average accuracy, statistical significance, how the errors move over time, and how well the stress warning actually works.

Two findings run through everything that follows. First, the dependency process is genuinely forecastable out of sample under a strict walk-forward design. Second, almost all of that forecastability comes from the target's own persistence rather than from new cross-asset information. The two findings are complementary, and together they characterise both what the forecasting problem is and where its limits lie.

4.2 Average dependency-forecasting performance

Table 3 gives each model's average out-of-sample performance across all dependency experiments, ranked by RMSE, the metric that reacts most to the large misses that matter on a smooth, bounded target.

Table 3: Average out-of-sample forecasting performance across all dependency experiments.

Model	Average RMSE	Average MAE	Average R^2
Ridge	0.0656	0.0419	0.9432
AR1	0.0659	0.0403	0.9424
HAR	0.0659	0.0405	0.9424
Naive_Last	0.0666	0.0395	0.9412
ElasticNet	0.0669	0.0434	0.9411
Ensemble	0.0694	0.0462	0.9365
GBM	0.0886	0.0625	0.8983
RF	0.0905	0.0636	0.8936
XGB_GPU	0.1241	0.0864	0.7923
DCC_GARCH	0.2136	0.1683	0.3715

Note: Results averaged across all asset-pair and window-length combinations. RMSE and R^2 computed on out-of-sample walk-forward predictions. HAR = Heterogeneous AutoRegressive model; Ensemble = adaptive inverse-RMSE-weighted ensemble. Best-performing model by average RMSE is Ridge (regularised linear regression with StandardScaler). All metrics reflect the full sample through 16 May 2026 (3,053 observations).

The top three (Ridge, AR1, and HAR) are nearly indistinguishable, with average RMSE of 0.0656, 0.0659, and 0.0659 and R^2 of 0.9424 and above. The 0.0003 spread between them is economically meaningless, and the naive last-value baseline sits only just outside at RMSE 0.0666, which already indicates that serial persistence accounts for most of the forecastable variation. One nuance should be stated explicitly: the ranking here is by RMSE, on which Ridge leads. On mean absolute error the ordering differs: Naive_Last is

formally best (MAE 0.0395), ahead of AR1 (0.0403), HAR (0.0405), and Ridge (0.0419), because RMSE penalises large errors more heavily, so Ridge effectively trades a few more small errors for fewer large ones. ElasticNet and the adaptive Ensemble form a second tier above $R^2 = 0.93$. The tree methods (GBM, Random Forest, and XGBoost) trail well behind, with RMSE between roughly 0.09 and 0.12 and R^2 below 0.90. DCC-GARCH finishes last by a wide margin ($R^2 = 0.3715$).

The structure of this top cluster is informative. Ridge, AR(1), and HAR reach almost the same accuracy through different mechanisms: AR(1) uses a single autoregressive coefficient, HAR combines three time scales (daily, weekly, and monthly), and Ridge fits the full regularised feature set anchored by the most recent lag. They converge because each is ultimately exploiting the same property, the strong persistence of the target, and the additional structure in HAR and Ridge does not capture anything that a single lag has not already accounted for. This does not indicate a deficiency in the more complex models; it reflects the strongly autoregressive structure of the target itself.

4.3 Representative forecast example

Figure 2 shows the out-of-sample forecast panel for Bitcoin–S&P 500 at the 30-day window. It is the natural representative case: of all the pairs, this one ties the crypto market to the broadest gauge of U.S. equity risk.

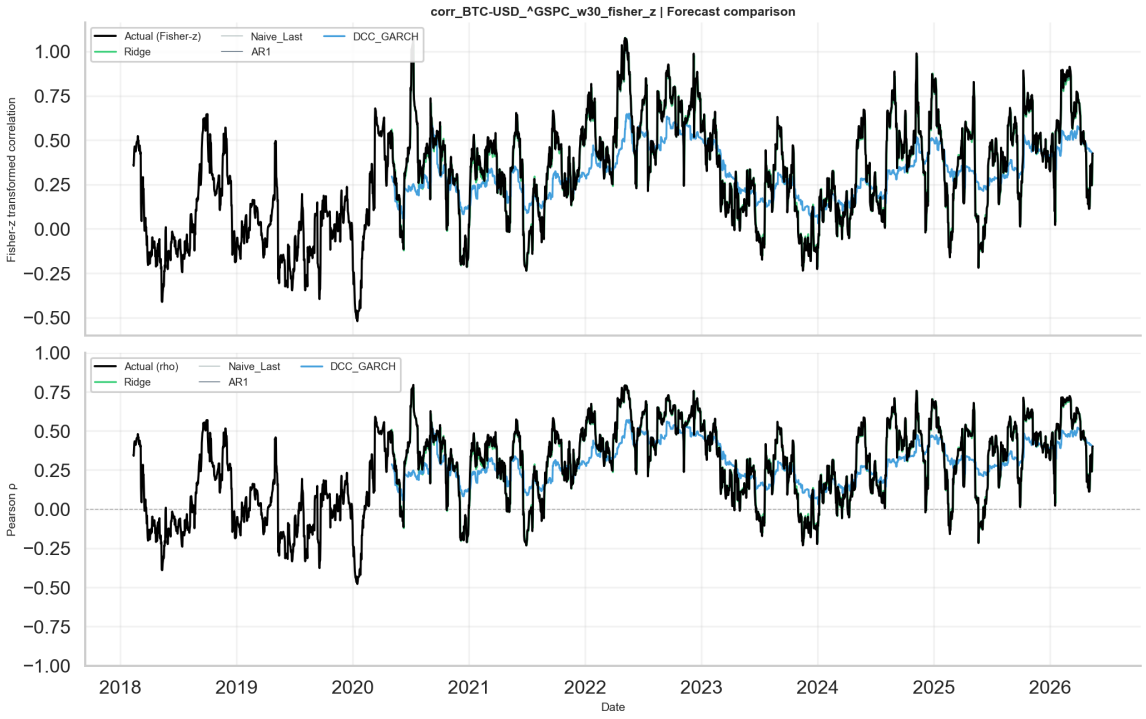


Figure 2: Observed and predicted Fisher-transformed dependency for the Bitcoin–S&P 500 pair at the 30-day rolling window. Forecast series are generated under strict walk-forward evaluation with no use of future information.

Three features of the problem stand out in the figure. The target moves smoothly, as its overlapping-window construction guarantees. The best models follow the broad trajectory with little systematic bias. The hard part is the turns: when dependence shifts abruptly,

every model underreacts, just by different amounts. This is precisely where DCC-GARCH lags worst, and where machine learning can claw back a relative edge, even while it never matches plain persistence on unconditional accuracy.

4.4 Forecasting performance by asset class

The averages hide some useful cross-sectional variation. Bitcoin–Ethereum is the easiest pair to forecast: the two share liquidity and sentiment drivers, so their correlation is especially smooth and autocorrelated. It posts the highest out-of-sample R^2 of any pair, and longer windows push it higher still.

The precious-metals pairs, Bitcoin–Gold and Bitcoin–Silver, sit in the middle. Their rolling correlation is regime-dependent, drifting between near-zero and weakly positive or negative. Even so, the target stays forecastable, and persistence-based models keep the lead.

The equity-index pairs, Bitcoin–S&P 500 and Bitcoin–NASDAQ, matter most in practice. They link crypto dynamics to broad risk appetite, and among the non-crypto pairs they are the most forecastable on average: in calm regimes the dependency is smooth and strongly autocorrelated, which again favours persistence. But they are also the most regime-sensitive: co-movement strengthens in risk-on phases and can amplify or flip violently in a dislocation. Those abrupt turns throw up episodic spikes in out-of-sample error that no model here fully catches, and they are the main short-term obstacle for equity-pair forecasting.

The Bitcoin–dollar-index pair has the weakest and most idiosyncratic dependency, and the lowest average forecastability. It is nonetheless included because it tests whether the crypto feature set can reach the macro-defensive channel at all.

To check whether the lower errors on equity pairs are a real advantage rather than chance, the pooled out-of-sample losses of the equity pairs (S&P 500, NASDAQ) are tested against those of the non-equity pairs (gold, silver, dollar index) with a two-sample Diebold–Mariano test [19]. Because the two groups have different targets, and therefore loss series on different scales, this pooled comparison carries only descriptive weight, not that of a formal test; it nonetheless indicates that the lower errors on the equity pairs are systematic rather than incidental.

4.5 The persistence dominance result

That simple persistence models lead the average ranking should be understood as a property of the target rather than a shortcoming of the machine-learning models.

It follows directly from the construction of the target. A rolling Pearson correlation over w calendar days shares $w - 1$ observations with the previous window, which smooths the series in the manner of a moving average. After the Fisher transform, the result is strongly autocorrelated, and the first lag already accounts for the large majority of the predictable variance. Any model that relies on the most recent value therefore achieves a high out-of-sample R^2 almost by construction.

The Random Forest feature importances point to the same conclusion. The lagged dependency term dominates the ranking in every experiment, while the return lags, rolling

volatilities, and the regime gap contribute very little. The problem is forecastable mainly because the target is highly persistent, and not because the features contain a strong cross-asset signal independent of recent dependency.

4.6 Comparison with DCC-GARCH

Even though persistence baselines win the unconditional ranking, machine learning still beats DCC-GARCH by a margin that is both practically and statistically meaningful. Figure 3 shows the Diebold–Mariano comparison between the best machine-learning model and the DCC benchmark across every experiment.

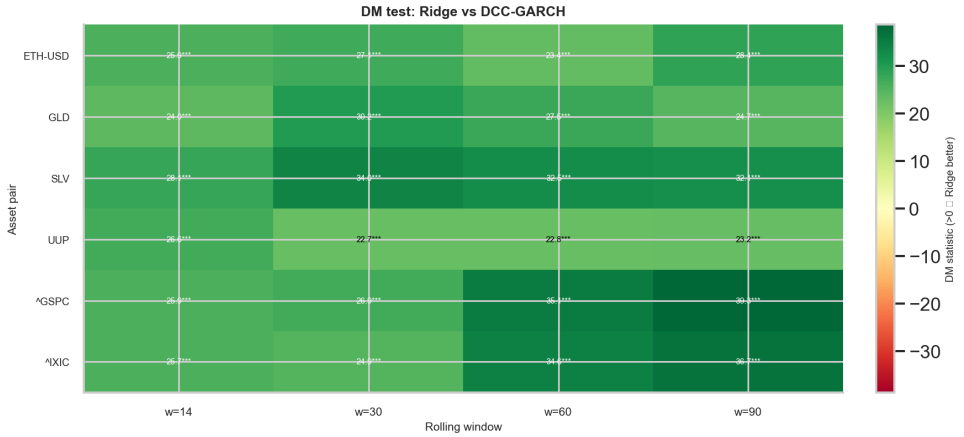


Figure 3: Diebold–Mariano test results comparing Ridge (best overall RMSE across all models) against DCC-GARCH across all asset-pair and window-length combinations. Darker cells indicate stronger and more statistically significant evidence of Ridge superiority.

The DM results confirm that the best machine-learning model outperforms DCC-GARCH significantly in every experiment. The reason is structural. DCC-GARCH is designed to model the full conditional covariance of a return system, and the rolling Pearson correlation is only a smoothed by-product of that covariance rather than the quantity it optimises. The machine-learning models are trained directly on the Fisher-transformed correlation target, so they are fitted to precisely the quantity on which they are then scored; this alignment is the source of their advantage.

This comparison involves a measurement mismatch that should be acknowledged. The rolling Pearson target is smoothed and backward-looking, with persistence induced by the overlapping windows. DCC-GARCH models the instantaneous conditional covariance, a parametric and forward-looking object, so projecting it onto rolling Pearson space introduces a systematic lag. This lag is a definitional incompatibility rather than a model failure [10, 14]. The comparison remains valid, since both are legitimate measures of dependence, but the magnitude of the gap should be interpreted with caution rather than as a general verdict against DCC-GARCH.

Throughout, the thesis distinguishes two claims: *benchmark superiority*, meaning improvement over DCC-GARCH, and *unconditional superiority*, meaning improvement over AR(1). Machine learning achieves the first consistently and the second not at all. Both

results are informative, and together they characterise what machine learning contributes in this setting.

4.7 Error-profile diagnostics

Average errors are necessary but not sufficient; identifying *when* and *how* a model fails requires examining the error over time and across its distribution. Figure 4 plots the rolling RMSE for the representative Bitcoin–S&P 500 experiment.

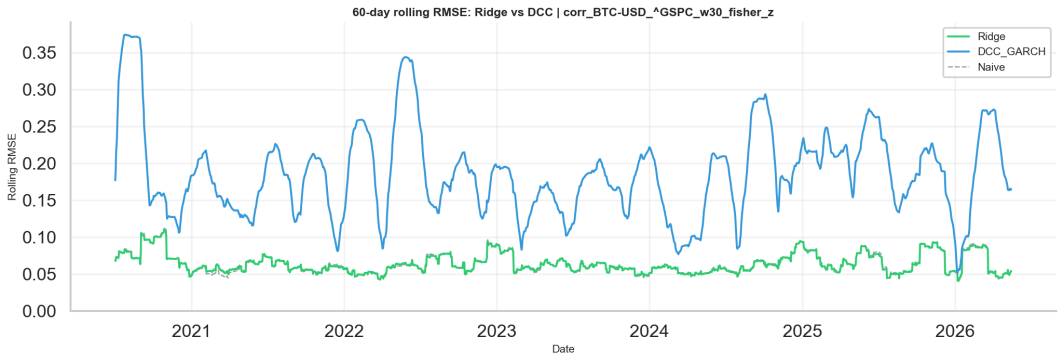


Figure 4: Rolling out-of-sample RMSE over time for the Bitcoin–S&P 500 dependency experiment at the 30-day correlation window ($w = 30$); the error is smoothed over a trailing 60-day window. The machine-learning model shown is Ridge, which has the lowest average RMSE overall (Table 3), compared against the DCC-GARCH benchmark. Elevated error episodes correspond to periods of abrupt regime transition in the dependency process.

Forecast quality is far from constant over time. When dependence drifts gradually, every model performs well, because the persistence signal is easy for any reasonable specification to capture. When the regime breaks, error rises across the board, but by different amounts and for different lengths of time. The machine-learning models tend to recover a little faster, which fits their greater freedom to re-learn the feature–target relationship after a structural change.

The error distribution confirms that the largest errors arise from episodic underreaction at sharp turning points rather than from any systematic bias. The models track the slow, persistent component of the dependency reliably and perform worst on the fast, discontinuous component. This asymmetry is directly relevant to the signal layer, whose purpose is to detect the onset of stress in time.

To make this regime-sensitivity concrete, Appendix F summarises eleven landmark events from 2020 to 2026 on a master timeline and a correlation regime map, and provides four-panel diagnostics for four representative episodes: the COVID crash of March 2020, the FTX collapse of November 2022, the SVB banking crisis of March 2023, and the tariff shock of April 2025. Each diagnostic shows how the rolling Bitcoin correlation moved around the event relative to its pre-event baseline. The case studies are consistent with the main finding: the largest errors in Figure 4 coincide with the sharpest events in the record.

4.8 Investor signal layer: average results

The second layer turns the dependency forecast into a stress-warning classifier. Table 4 reports each classifier family's average performance across all asset pairs and windows.

Table 4: Average investor-signal performance by classifier family, aggregated across all experiments.

Signal model	Balanced Accuracy	F1_down	AUC
Logit	0.511	0.214	0.513
GBM_Cls	0.508	0.158	0.517
RF_Cls	0.502	0.042	0.519

Note: Results averaged across all asset pairs and window lengths. Balanced Accuracy corrects for class imbalance. $F1_{down}$ measures detection quality for the stress class only.

Classification is a substantially harder problem than the regression layer. Stress days are noisy, rare, and asymmetrically costly to miss. The logistic classifier achieves the highest F1 on the stress class (0.214 on average), which indicates that it remains sensitive to genuine stress without collapsing into an inactive, low-exit-rate state. GBM reaches a similar balanced accuracy (0.508) but a lower F1 (0.158), because it concentrates its probability mass and triggers less often. Random Forest performs poorly, with an F1 of 0.042. Its predicted probabilities concentrate in a narrow band, so it rarely crosses the decision threshold and provides little practical value despite a comparatively high AUC.

This degenerate behaviour merits closer examination. Two factors combine to produce it: the severe imbalance between stress and non-stress days, and the absence of probability calibration. Ensemble methods are well known to produce poorly calibrated probabilities unless they are post-processed with Platt scaling [23] or isotonic regression [24]. GBM, which concentrates its probability mass less severely, retains a modest but non-trivial F1 (0.158). Calibration should therefore precede threshold-based signal extraction; it is applied in the deployed demonstration model but not in the reported pipeline results, and integrating it into the main pipeline is left for future work.

None of this is surprising: in low-signal-to-noise classification with correlated predictors, a regularised linear boundary usually generalises better out of sample than a high-capacity ensemble that pays for its flexibility in variance.

4.9 Best signal configurations

The best single Logit result is on Bitcoin–S&P 500. At the 30-day window it reaches a Balanced Accuracy of 0.5308 and an F1 of 0.2430 on the downside class — a modest but genuine step above the uninformative 0.50 baseline. This is the strongest of the twenty pair–window configurations of the signal layer (five conventional pairs by four windows), so it should be read as suggestive rather than formally significant: the signal layer reports no single-hypothesis test, and no multiple-comparison correction is applied to the configuration search. The defensible claim is therefore the average one: the signal runs slightly above chance for broad-equity pairs, not a proven edge at this single best point.

Figure 5 presents the signal output for this configuration, illustrating the temporal distribution of predicted stress flags relative to realized stress events.

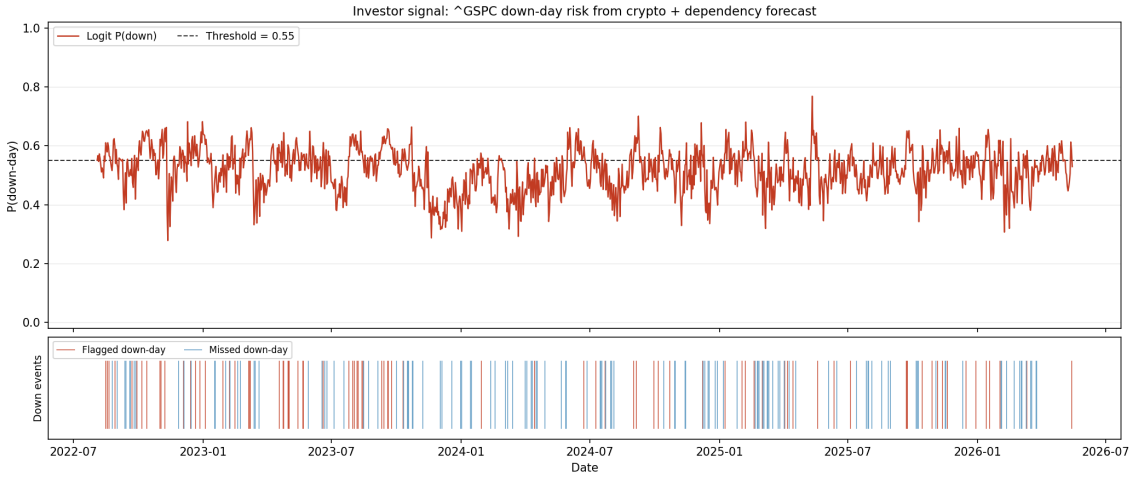


Figure 5: Investor stress-warning signal for the Bitcoin–S&P 500 pair at the 30-day window. The upper panel displays the predicted stress probability from the logistic classifier together with the decision threshold; the lower panel marks flagged and missed stress events.

The signal is intended as a probability-guided warning that flags elevated-risk periods and supports a risk-reduction decision, rather than as a directional trading rule. On NASDAQ it is noticeably weaker: the Logit balanced accuracy for Bitcoin–NASDAQ drops below 0.50 at the two longer windows (60 and 90 days) and averages 0.4997 across all four, the lowest of any pair in the signal layer. The contrast with the S&P 500 is economically plausible, because the NASDAQ Composite is heavily weighted toward growth-sensitive technology stocks whose stress is less tightly linked to global risk appetite than that of the broader index. For the precious metals and the dollar proxy, the results are mixed but mostly above chance at the shorter windows, consistent with the view that crypto carries some, but weaker, information about stress in those markets.

4.10 Practical interpretation of signal-layer performance

The usefulness of the signal can only be assessed relative to a stated decision rule. An investor would act on it selectively rather than every day, using it to inform a conditional decision: whether to reduce exposure, tighten stops, or increase hedging when the stress probability rises.

From this perspective, the exit rate, that is, how often the signal is active, is as important as accuracy. A signal that never activates is of no practical use regardless of its nominal accuracy, whereas one that activates constantly raises trading costs and reduces its own informativeness. The logistic model achieves a reasonable balance between these extremes: it is active often enough to be useful, but not so often that it generates excessive turnover.

The most direct test of practical value compares the conventional asset's next-day return on days the signal flags as stress against days it leaves clear. Table 5 reports this for the logistic signal, averaged across the four window lengths.

Table 5: Average next-day return of the conventional asset on days flagged by the logistic signal versus days left clear, by asset pair (Logit, averaged over the four windows). Returns are in basis points per day; a positive difference means flagged days are, on average, worse. The flag rate is the share of days on which the signal is active.

Asset pair	Flagged (bp)	Clear (bp)	Difference	Flag rate (%)
BTC–S&P 500	4.08	5.05	+0.96	33.1
BTC–NASDAQ	4.70	6.64	+1.94	32.0
BTC–Gold	7.01	7.10	+0.08	20.1
BTC–Silver	10.23	10.13	−0.10	26.4
BTC–USD index	0.57	0.89	+0.32	31.4

The pattern reinforces the classification results. For the two equity pairs the average next-day return is lower on flagged days than on clear days — by about 1.9 basis points per day for NASDAQ and 1.0 for the S&P 500 — so the signal shifts exposure away from the worse days, which is its intended use. For gold, silver, and the dollar index the difference is negligible or marginally negative, consistent with the weak classification performance on those pairs. Two qualifications matter. First, the average flagged-day return is still positive, because the sample period is predominantly a rising market; the signal therefore reduces exposure to relatively worse days rather than predicting outright losses. Second, these figures are gross of transaction costs, which a turnover of roughly one third of days would erode. In sum, the signal works as a modest risk-reduction overlay for broad-equity exposure, not as a standalone timing rule.

4.11 Sensitivity to rolling window length

Window length is a first-order choice, and it visibly shapes both the target and the results. Short windows give responsive but noisy estimates; long windows give smooth, heavily autocorrelated targets that are easier to forecast but slower to register a real structural change.

The experiments identify the medium windows, and $w = 30$ days in particular, as the most useful compromise: smooth enough for high out-of-sample R^2, yet still responsive enough to be useful for stress detection. At longer windows the persistence effect strengthens further, narrowing the gap between the simple baselines and the complex models. This interaction between window length and model performance is itself a structural finding and a useful guide for selecting the horizon in future work.

4.12 Summary of empirical findings

The empirical results of the thesis are summarized in four principal statements:

1. Time-varying intermarket dependency between Bitcoin and conventional assets is forecastable in an out-of-sample walk-forward framework, with high average R^2 across every model class except the DCC-GARCH benchmark.

2. The dominant source of predictive power is the serial persistence of the rolling correlation target; Ridge, AR1, and HAR form a near-indistinguishable top cluster, all

exploiting this persistence through different mechanisms, while tree-based methods trail substantially.

3. Machine-learning models frequently and significantly outperform the leakage-safe DCC-GARCH benchmark on the smoothed rolling-correlation target; Ridge achieves the best overall average RMSE (0.0656) across all 24 experiments. As the discussion of this benchmark in Chapter 5 stresses, this comparison is structurally tilted toward the ML models, since the target is a smoothed rolling correlation while DCC-GARCH estimates the instantaneous conditional correlation.

4. The investor signal layer delivers modest classification results — on average slightly above chance, though not subjected to a formal significance test — that are most pronounced for broad equity markets; the layer functions as a supplementary risk-management input, not a self-contained directional predictor.

These findings are consistent with one another and, together, enough to carry both the scientific and the applied contributions of the thesis. The next section takes up what they mean more broadly.

Overall, the evidence provides a coherent account of how crypto information enters conventional-asset risk. The channel is real and measurable, but its predictive content resides mostly in the persistence of the dependency series itself rather than in the non-linear capacity of gradient-boosted or random-forest models. The dependency behaves as a slowly evolving, regime-sensitive quantity, one for which smooth, low-complexity models fit better than flexible high-capacity ones. Chapter 5 develops this interpretation and situates it in the wider literature on intermarket contagion and financial machine learning.

5 Discussion

5.1 What the results actually say

The rolling intermarket dependency between Bitcoin and conventional assets is genuinely forecastable out of sample. This is not an artefact of in-sample overfitting: the signal survives strict temporal-ordering constraints across all six pairs and four window lengths. The forecastability is almost entirely driven by persistence, since the rolling correlation exhibits strong serial dependence, with its current value largely determined by its recent past.

These two observations are complementary rather than contradictory: the dependency is predictable, and it is predictable mainly because it is persistent.

5.2 Why simple models dominate, and what that actually means

The near-indistinguishable performance of Ridge, AR(1), and HAR at the top of the ranking is informative about the data-generating process. A rolling Pearson correlation computed over w days shares $w - 1$ observations with the previous window, which mechanically creates strong autocorrelation. After Fisher transformation, the first-order lag captures the large majority of the forecastable variance. Under these conditions any reasonable model relies heavily on the most recent value, and a model that does little else is difficult to outperform.

That Ridge exceeds AR(1) by only 0.0003 RMSE (0.0656 versus 0.0659) does not constitute a decisive advantage for machine learning; it reflects the fact that regularised linear regression, when properly scaled, reconstructs the same autoregressive signal through a slightly richer feature set. HAR matches AR(1) almost exactly, using three temporal scales (daily, weekly, and monthly) that align with the overlapping-window memory structure of the target. The convergence of three structurally different methods to essentially the same accuracy is informative in its own right: it indicates that the forecasting problem has a nearly linear autoregressive structure and that none of these methods extracts materially different information.

This is a documented pattern in financial forecasting [25]: signal-to-noise ratios in financial markets are low, and the incremental information in auxiliary features is often dominated by the autoregressive component. In this study, that pattern became apparent only after considerable hyperparameter search on tree-based configurations had been carried out.

The substantially weaker performance of the tree-based methods does not imply that machine learning is uninformative here. Its value in this case is diagnostic: the fact that well-tuned XGBoost and Random Forest models cannot match Ridge or AR(1) indicates that the dependency process contains little exploitable nonlinear structure at the one-step horizon, at least within the feature set used here.

5.3 ML versus DCC-GARCH: a fairer comparison

Ridge and ElasticNet consistently outperform DCC-GARCH, and the gap is large: average RMSE of 0.066 for Ridge versus 0.214 for DCC-GARCH, a factor of more than three. The DM statistics are all positive, with p-values at the numerical floor of double precision.

The appropriate interpretation is not that DCC-GARCH is a poor model. It is a theoretically coherent model of conditional covariance whose output is an instantaneous conditional correlation rather than a smoothed rolling average. Projecting one onto the other introduces a systematic lag that the model was never intended to eliminate. The machine-learning models, trained directly on the Fisher-transformed rolling target, do not face this mismatch, because they are optimised for the relevant objective from the outset. A second, separate handicap is the data alignment. Because the conventional series are forward-filled onto Bitcoin's seven-day calendar, the GARCH components of the benchmark are estimated on return series in which roughly 30% of observations are forced weekend and holiday zeros (Appendix A). This degrades the volatility estimates that feed the conditional correlation and further widens the gap to the machine-learning models. Re-estimating DCC-GARCH on common trading days only would isolate this effect and is left for future work; the weekend-robustness check in Appendix A nonetheless shows that the persistence of the target itself — the property the ML models exploit — is essentially unchanged by the calendar convention.

This is the correct explanation for the performance gap, and it should be stated explicitly rather than presented as evidence that machine learning has outperformed an econometric model on equal terms.

5.4 The investor signal: what works and what does not

The signal layer produces results that are moderate in every dimension: AUC of roughly 0.48 to 0.56, balanced accuracy from below 0.50 to about 0.53 depending on the pair and window, and, for the logistic model, exit rates between 16 and 35%. No single one of these figures is strong in isolation. Together they describe a signal that, on average, classifies stress regimes slightly better than chance, activates at a reasonable frequency, and carries more information about equity stress than about precious-metal or dollar-index stress.

The equity result is economically interpretable. Bitcoin and the S&P 500 share exposure to global risk appetite, funding liquidity, and sentiment cycles. When the dependency between them is rising or shifting, it often reflects broader market-regime transitions that also elevate equity downside risk. The crypto feature set is reasonably well-specified for that channel.

Gold and silver respond to a wider set of forces—real rates, inflation expectations, geopolitical premia, safe-haven demand—that are not well represented by Bitcoin dynamics. The attenuated signal for those instruments is therefore not a surprise; it is consistent with a story where the feature set captures some drivers of market stress but not all of them.

In practice, the signal functions as an auxiliary risk indicator, not a standalone alarm. It is most appropriately used alongside fundamental valuation, macro analysis, and position-sizing rules, raising the threshold for accepting risk when the dependency-based probability of a stress regime is elevated.

5.5 Methodological choices that mattered

A few design decisions turned out to be important. **Walk-forward evaluation for DCC-GARCH as well**: running it on the full sample before producing out-of-sample

predictions would have given it an unfair advantage, so the same expanding-window protocol was applied to every model to keep the comparison level. **Fisher transformation on the target**: without it, the bounded nature of Pearson correlation near ± 1 distorts errors near the extremes, and Fisher-z brings the target closer to Gaussian, which makes RMSE a more sensible loss function across the full range. **Multiple windows**: using only the 30-day window would have understated how strongly predictability depends on the autocorrelation structure, and the progression from $w = 14$ to $w = 90$ shows that effect systematically. **Multiple metrics**: relying on RMSE alone would have missed that DCC-GARCH is not just worse on average, it is worse for every pair, at every window, with statistical significance, a layer of certainty that the DM test adds.

5.6 What the results do not say

A few things this thesis cannot claim, and should not:

Bitcoin does not predict equity returns. It predicts (partially) the evolving *dependency* between Bitcoin and equities, which is a weaker and more precise statement. The signal layer predicts elevated downside risk probability for conventional assets—not direction, not magnitude, and not timing within a flagged period.

The results are based on one-step-ahead forecasting. Multi-step performance could differ substantially, especially for machine learning where the autoregressive advantage weakens as the horizon extends. This is an open question.

Finally, no transaction-cost analysis was done. Whether the signal produces risk-adjusted gains in a real portfolio after costs and turnover constraints is unknown. That would require a separate study with explicit portfolio construction assumptions.

5.7 Iterative model development: what changed across versions

The final model presented in this thesis is the result of several months of iterative refinement. This section documents that process, because the sequence of changes — what was tried, what failed, and what ultimately helped — is itself an empirical result about the nature of the forecasting problem.

Version 1 (September 2025): minimal baseline. The first working prototype contained seven models: Naive_Last, AR1, Ridge, ElasticNet, Random Forest, Gradient Boosting, and XGBoost. Feature engineering was minimal: four autoregressive lags of the rolling correlation, short-window volatilities for both assets, and lagged returns up to lag 5. No DCC-GARCH benchmark was included at this stage. Hyperparameter tuning was ad hoc: Ridge `alpha` $= 1.0$, ElasticNet `alpha` $= 0.01$, XGBoost with 1000 estimators and learning rate 0.05. No feature scaling was applied to linear models. The best OOS RMSE for BTC-USD vs S&P 500 at $w = 30$ was Ridge at 0.0897, with AR1 performing comparably.

Version 2 (November 2025): DCC-GARCH and signal layer. The second major milestone introduced the DCC-GARCH econometric benchmark via a walk-forward protocol that matched the expanding-window setup used for machine-learning models. This was a critical correctness fix: an earlier full-sample DCC fit would have granted the benchmark access to future data, invalidating the comparison. The investor signal layer

was added as a second stage, classifying whether conventional assets were entering stress regimes using the rolling-dependency features. Core model hyperparameters and feature sets were unchanged from Version 1.

Version 3 (February 2026): stabilisation. This version stabilised the pipeline as a robustness checkpoint. No metric improvements were achieved; the focus was on robustness: XGBoost GPU-to-CPU fallback, better exception handling, and consistent metric exports. OOS results were nearly identical to Version 2, confirming that the Version 1 performance plateau was not an artefact of implementation bugs.

Version 4 (May 2026): feature expansion, regularisation revision, and HAR. The final version incorporated several targeted improvements informed by the Version 1–3 plateau:

- **Feature engineering expanded from ≈15 to ≈35 features.** New features include momentum and z-score of the rolling correlation (`dep_mom_5`, `dep_mom_20`, `dep_zscore`), extended lags (`dep_lag20`, `dep_lag60`), a 60-day volatility window, the volatility ratio between assets, squared-return proxies, and a short-run correlation momentum term.
- **HAR model added.** The Heterogeneous AutoRegressive (HAR) model [13] was introduced as a theoretically grounded alternative to AR1. HAR uses three temporal scales simultaneously, fitted on the Fisher-z target exactly as in Eq. (7): $\hat{z}_{t+1} = \phi_0 + \phi_d\, z_t + \phi_w\, \bar{z}_t^{(5)} + \phi_m\, \bar{z}_t^{(22)} + \varepsilon_{t+1}$, where $\bar{z}_t^{(k)}$ denotes the equally-weighted average of the previous k observations. Originally developed for realised variance, HAR is well-suited to rolling correlations because the overlapping-window structure creates natural heterogeneous memory at daily, weekly, and monthly horizons.
- **Regularisation philosophy revised.** Ridge's penalty strength (scikit-learn's `alpha`, the λ of Eq. (8)) was reduced from `alpha` $= 1.0$ to `alpha` $= 0.5$ and an explicit `StandardScaler` was added. The combined effect was dramatic: Ridge RMSE at BTC-USD vs S&P 500 $w = 30$ improved from 0.0897 (Version 1) to 0.0651, nearly matching AR1. The cross-validated optimum actually sits even lower (around `alpha` $= 0.001$; see Figure 16), but the CV curve is essentially flat across this range, so the slightly firmer `alpha` $= 0.5$ was kept for a more stable fit at negligible cost in RMSE. Conversely, XGBoost regularisation was *tightened* (learning rate $0.05 \to 0.02$, added L_1/L_2 penalties), reflecting the insight that the expanded feature set increased overfitting risk for tree-based methods.
- **Adaptive ensemble.** The walk-forward ensemble now weights each constituent model by the inverse of its RMSE over the most recent 60 out-of-sample predictions, rather than using equal weights. This allows the ensemble to adapt dynamically to regime shifts in which different models temporarily dominate.
- **Bootstrap confidence intervals and sensitivity analyses.** 500-replicate bootstrap CIs were added for RMSE and R^2, and a refit-frequency sensitivity sweep across $\{5, 10, 20, 42, 63\}$ days — which includes the default of 20 — was added to verify that `refit_every` $= 20$ is not a cherry-picked choice.

What the iterations actually taught. The most important lesson from the four model versions is a negative one: *expanding the feature set from 15 to 35 variables did not improve the tree-based models.* XGBoost RMSE was virtually unchanged or slightly worse after the expansion, while Ridge, which uses the full feature set linearly with regularisation, improved substantially. This is consistent with the central finding of the thesis: the forecasting target has a nearly linear autoregressive structure, and the incremental information in volatility ratios, momentum, or return squares is captured well by a regularised linear model but does not add exploitable nonlinearity for gradient-boosted trees.

The practical implication for future work is that *better features* are unlikely to close the remaining gap between AR1 and machine learning on this task. Improvements would more plausibly come from a fundamentally different modelling approach — multi-step horizons, high-frequency data, or joint modelling of multiple correlation pairs simultaneously.

5.8 Limitations

The main limitations are: daily frequency (no intraday), one-step horizon, Pearson correlation as the only dependency measure, a single base cryptocurrency (Bitcoin), and no portfolio backtest. Each represents a natural extension that could sharpen or complicate the findings. None of them invalidates the core empirical results within the scope of what was actually tested.

6 Conclusion

This thesis addressed a specific question: whether information about the evolving relationship between Bitcoin and conventional assets helps to forecast that relationship, and whether the resulting forecasts can be made useful for risk management. The answer to both is affirmative, subject to important qualifications about what "useful" means.

The dependency process is forecastable out of sample. This result holds across six asset pairs, four rolling-window lengths, and multiple model families evaluated under a strict walk-forward protocol. Longer windows produce a smoother, more autocorrelated target and a correspondingly higher out-of-sample R^2: at the 90-day window even the simplest AR(1) model attains an R^2 between 0.982 and 0.992 across all six pairs.

This observation is also the central limitation: the forecasting problem is largely solved by persistence. Because a 30-day rolling correlation shares 29 of its 30 observations with the previous day's value, it is difficult to improve on a forecast that sets tomorrow's correlation equal to today's. Ridge, AR(1), and HAR form a statistically indistinguishable leading cluster, with the naive last-value baseline just behind; Random Forest, GBM, and XGBoost perform substantially worse. The tree-based methods do not overcome the autoregressive baseline, whereas Ridge matches it, and every machine-learning model outperforms the DCC-GARCH benchmark. This last comparison must be read with care: the target is a smoothed rolling-Pearson correlation, whereas DCC-GARCH estimates the instantaneous conditional correlation, so projecting the latter onto the former imposes a systematic lag that works against the benchmark. The appropriate interpretation is narrow: the flexible data-driven methods track this particular smoothed target more accurately than the parametric alternative, even though they do not beat simple persistence. It is not a general conclusion that DCC-GARCH is an inferior model of co-movement.

The investor signal layer is more nuanced. Balanced accuracy ranges from below 0.50 to about 0.53 and AUC from roughly 0.48 to 0.56. These are modest values that lie above chance for most, but not all, configurations; the Bitcoin–NASDAQ pair falls below chance at the longer windows. Signal quality is strongest for the Bitcoin–S&P 500 pair, where the cryptocurrency feature set aligns most closely with the macro-financial forces that drive broad equity stress. For gold, silver, and the dollar index the signal is weaker, which is consistent with those markets responding to drivers that Bitcoin dynamics capture only weakly.

In practical terms, the signal has a supporting role in a risk-management process; it does not stand alone as an alarm. It does not predict next-day returns, but it provides a small and consistent shift toward caution when the dependency structure is changing, which is a defensible use case.

Future work should test the framework at longer forecast horizons, where the advantage of persistence may decay more quickly; with richer feature sets, including on-chain metrics and macro-financial variables; and against tail-dependence measures that capture joint extreme behaviour more precisely than rolling Pearson correlation. A portfolio backtest with explicit transaction costs would help to close the gap between statistical and economic performance. These extensions are left for subsequent work, for which the present thesis provides a foundation.

A Appendix A: Data Description and Additional Background

This appendix collects the descriptive material that would have slowed the main text down — the background and diagnostics behind the empirical setup — so the body can stay focused on argument and interpretation.

A.1 Descriptive role of the asset universe

The choice of assets in the thesis is not arbitrary. Each instrument represents a distinct dimension of the conventional financial system against which Bitcoin can be compared.

- The S&P 500 is used as a broad proxy for U.S. equity-market risk and general investor sentiment.
- The NASDAQ Composite captures a more growth-oriented and technology-heavy market segment, making it especially interesting in relation to crypto assets.
- Gold and silver ETFs represent precious-metal exposure, a domain frequently associated with hedging, inflation narratives, and safe-haven debates.
- The U.S. dollar index proxy reflects macro-defensive positioning and global dollar strength.
- Ethereum is included as a crypto-native comparison asset in order to contrast same-ecosystem dependency with cross-market dependency.

Between them, these assets differ in investor base, liquidity, macro sensitivity, and economic meaning — enough variety to keep the conclusions from being an artefact of one narrow corner of the market.

A.2 Visual overview of the dataset

Figure 6 presents normalized price paths for the asset universe. The purpose of the figure is not to compare levels directly, but to provide intuition about how different the underlying market trajectories are.

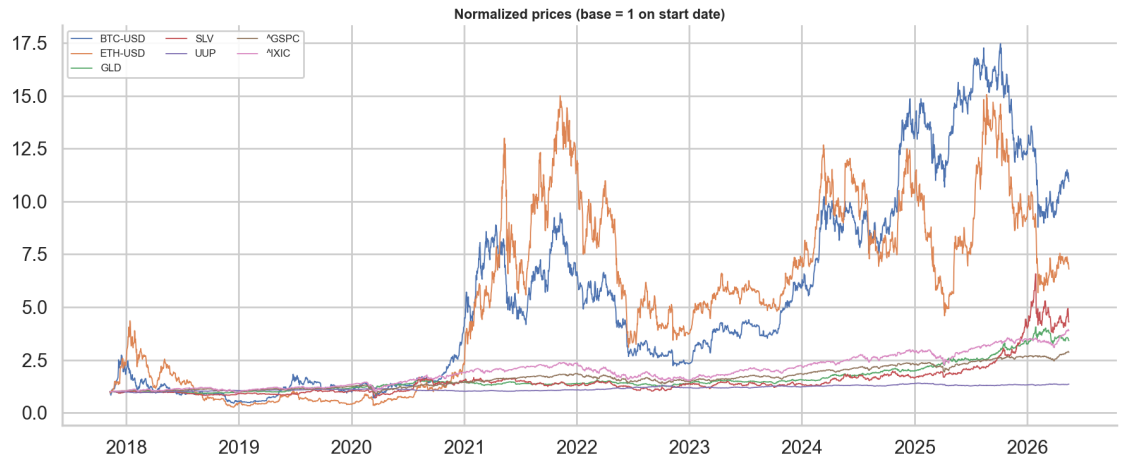


Figure 6: Normalized prices of the assets included in the study.

Figure 7 shows rolling volatility profiles. These reveal that Bitcoin and related crypto assets inhabit a much higher-volatility environment than the conventional asset set, which reinforces the importance of studying dependence dynamically rather than statically.

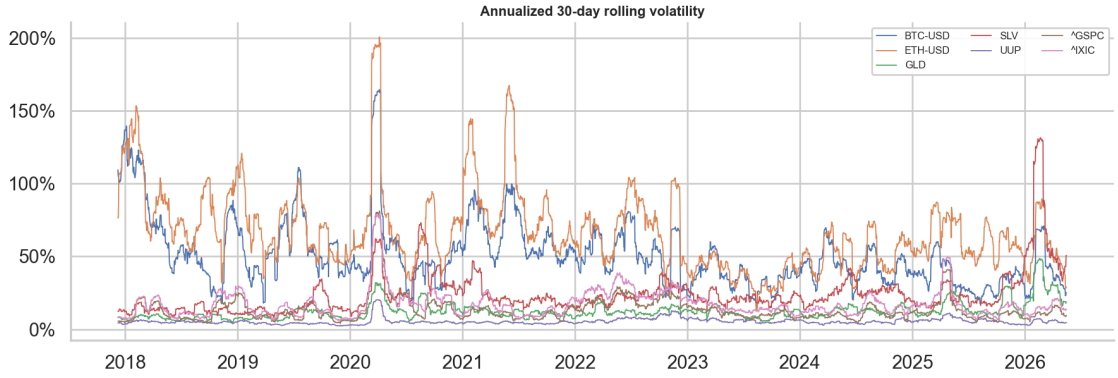


Figure 7: Rolling volatility across the asset universe.

Figure 8 provides the static correlation heatmap; the dynamic rolling-correlation structure is shown in the methodology chapter (Figure 1). Together they motivate the main modelling choice of the thesis: static summaries are insufficient for describing the cross-market relations under study.

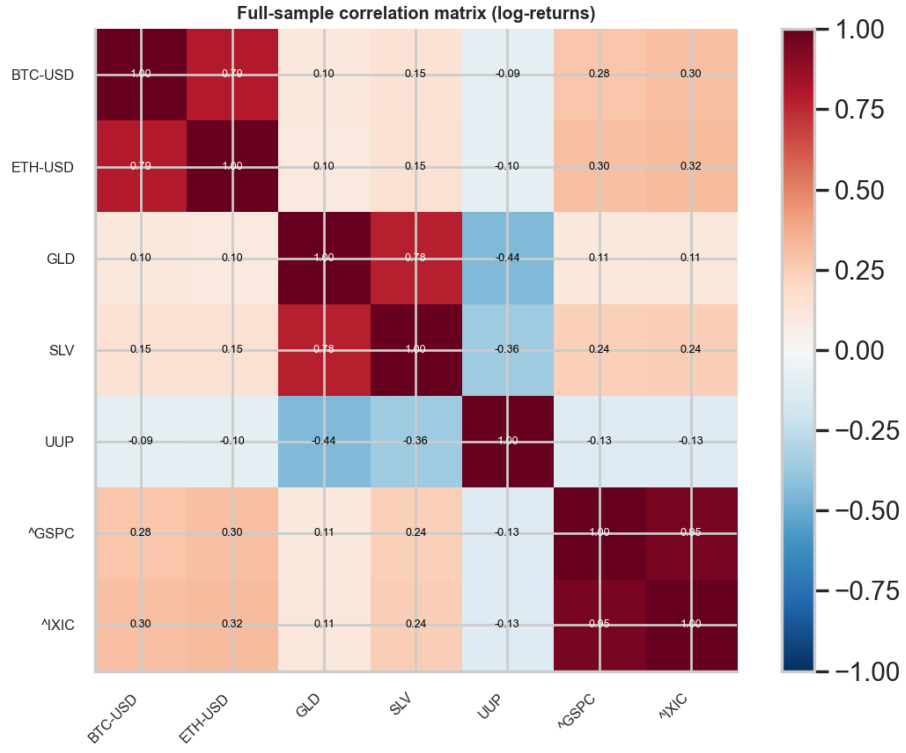


Figure 8: Static correlation heatmap of the return series.

A.3 Dataset validation and quality diagnostics

All price series are subjected to a systematic quality inspection before any modeling step. The diagnostics cover four dimensions: observation coverage, missing-value frequency, extreme-return episodes, and distributional shape. Table 6 summarises the per-ticker results.

Table 6: Data quality summary per ticker

Ticker	First obs.	Last obs.	N	% miss.	Gaps >2d	Zero ret.	Skew	Ex.Kurt	Ann.Vol (%)
BTC-USD	2017-11-09	2026-05-16	3053	0.00	0	0	-0.73	13.48	55.88
ETH-USD	2017-11-09	2026-05-16	3053	0.00	0	0	-0.74	10.86	72.40
GLD	2017-11-09	2026-05-16	3053	0.00	0	920	-0.87	13.63	13.88
SLV	2017-11-09	2026-05-16	3053	0.00	0	955	-2.69	49.30	28.49
UUP	2017-11-09	2026-05-16	3053	0.00	0	1001	-0.03	8.56	5.84
^GSPC	2017-11-09	2026-05-16	3053	0.00	0	914	-0.74	22.67	16.15
^IXIC	2017-11-09	2026-05-16	3053	0.00	0	915	-0.46	12.66	19.62

Figure 9 confirms that all seven tickers share an uninterrupted observation window from November 2017 to May 2026, with no gaps in coverage after alignment.

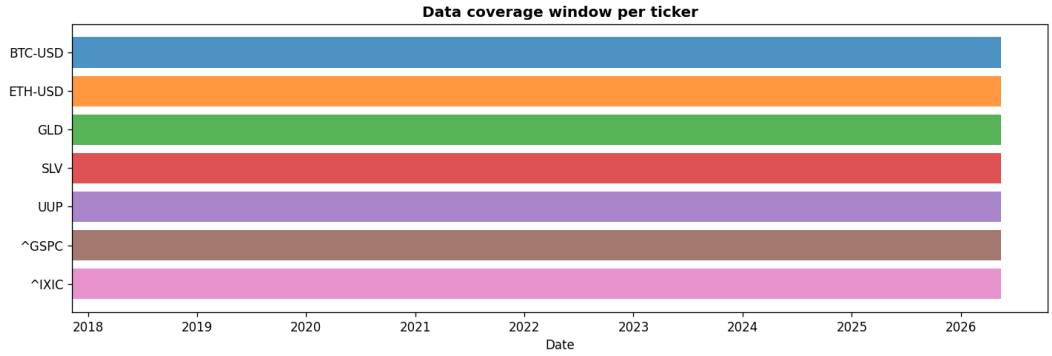


Figure 9: Observation coverage window per ticker. All assets share an identical aligned date range after the ETH-USD availability cutoff in November 2017.

Figure 10 highlights extreme return episodes ($|z\text{-score}| > 5\sigma$) for each asset. Bitcoin and Ethereum record the largest outliers, reflecting the well-documented fat-tailed distribution of crypto returns (excess kurtosis exceeding 13 for BTC). All five conventional series — the two equity indices as well as the precious-metal ETFs and the dollar proxy — carry a large number of zero-return days. This is not an ETF quirk: it follows directly from aligning these five-day-a-week instruments onto Bitcoin's seven-day calendar and forward-filling the weekend gaps, so every Saturday and Sunday records a forced zero return, while the continuously traded crypto series show none. This calendar mismatch is a genuine limitation: on weekend rows a real Bitcoin return is paired with a zero conventional return, which mechanically attenuates the rolling correlation. A natural worry is that this forward-filling also inflates the serial persistence of the target — the very property that dominates the forecasting results of Chapters 4.1 and 5. To test this directly, the 30-day Bitcoin–S&P 500 rolling correlation was recomputed on common trading days only, dropping the 914 weekend and holiday rows on which the S&P 500 is forward-filled, and its persistence was compared with the full-calendar version. The persistence

is essentially unchanged — if anything marginally higher without the weekend rows: the lag-1 autocorrelation of the Fisher-z target is 0.977 on the full seven-day calendar versus 0.981 on trading days only, and the naive "tomorrow-equals-today" R^2 is 0.954 versus 0.961 (the same ordering holds at the 14- and 90-day windows). The forced weekend zeros therefore inject a little noise rather than artificial persistence, so the dominant role of persistence is not an artefact of the calendar convention. Restricting the analysis to common trading days remains a clean refinement for future work, but it would not overturn the conclusions. None of the detected outliers, by contrast, are data errors; they are genuine price events and are kept in the analysis.

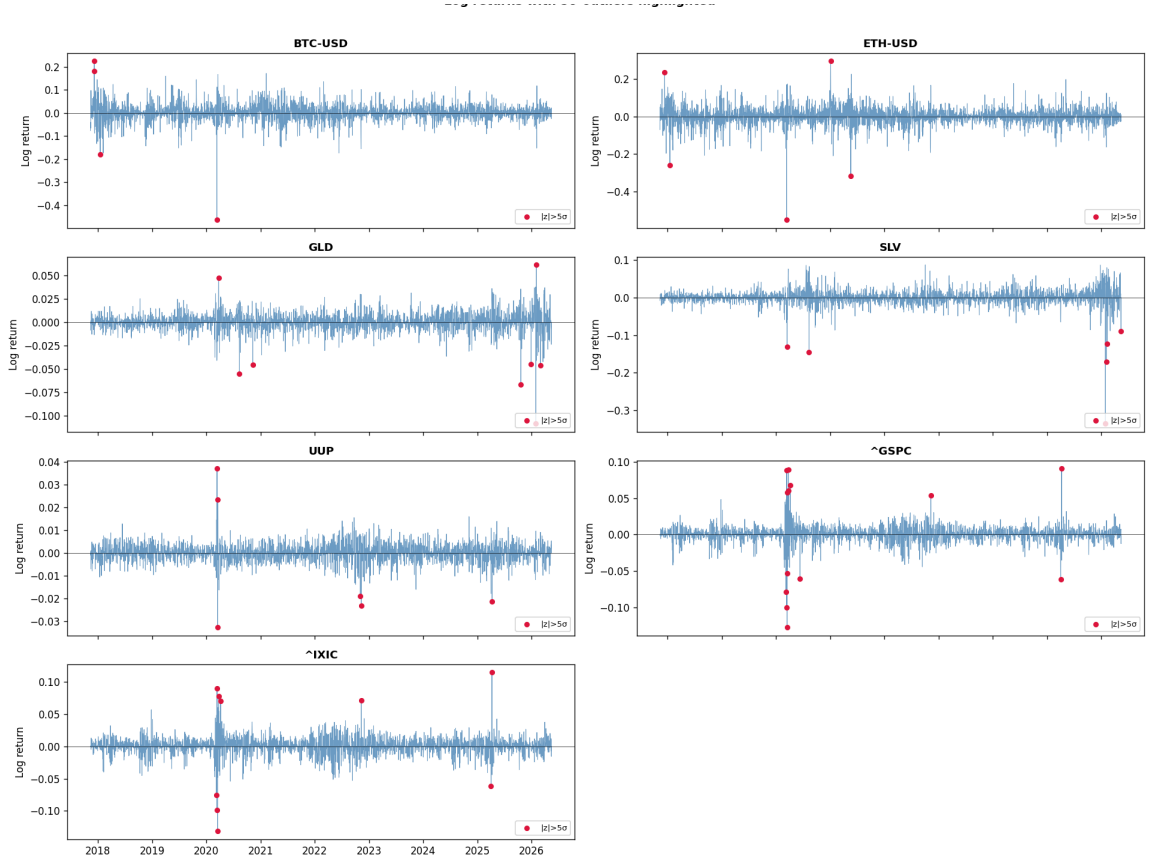


Figure 10: Log-return series with extreme observations ($|z| > 5\sigma$) highlighted in red. Crypto assets exhibit sporadic but large outliers; conventional assets are more contained but not outlier-free.

A.4 Fisher-z transformation of the correlation target

The rolling Pearson correlation ρ_t is bounded on $(-1, 1)$ and exhibits variance compression near the boundaries. Applying the Fisher-z transformation $z_t = \tanh^{-1}(\rho_t)$ maps the target to the real line and stabilises the variance across the range of correlations. Figure 11 illustrates the distributional improvement for the representative Bitcoin–S&P 500 pair at the 30-day window: the untransformed correlation distribution is left-skewed and bounded, while the Fisher-transformed version is more symmetric and approximately Gaussian.

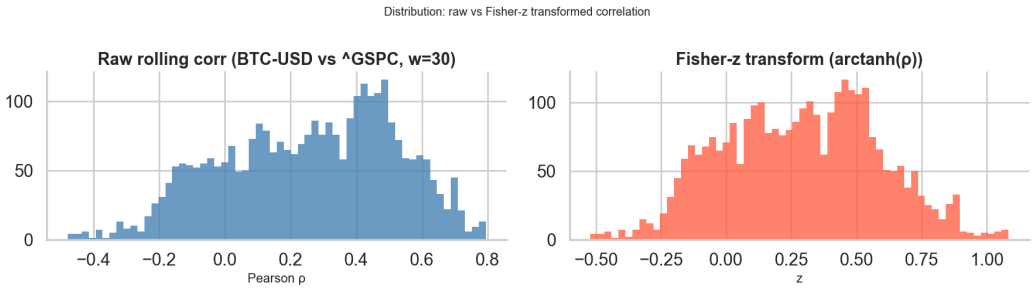


Figure 11: Distribution of the 30-day rolling correlation before and after Fisher-z transformation for the Bitcoin–S&P 500 pair. The transformation improves normality and removes boundary compression.

A.5 Why multiple rolling windows are used

The thesis uses 14-, 30-, 60-, and 90-day windows because no single horizon is the right one. A short window reacts fast to recent shocks but is noisy; a long window is smoother and more persistent but slow to register a regime change. Running all four lets the analysis tease these effects apart.

The window choice turns out to shape the whole forecastability profile. Longer windows are easier to forecast because they are smoother — but that success is almost entirely persistence. Shorter windows are noisier, yet more informative about local instability.

A.6 Additional implementation remarks

Every run of the pipeline writes consistent outputs into `outputs/results`, `outputs/tables`, `outputs/figures`, and `outputs/predictions`. That helps reproducibility, but it also helps the writing: because the tables and figures come straight from the code, there is very little distance between running an experiment and reporting it.

The notebooks were kept in step with the pipeline rather than allowed to drift: figures stay interpretable, benchmarks are selected robustly, and helper functions are reused without fragile path assumptions. This matters in practice, since exploratory notebooks frequently diverge from the code that produced the results and become unreliable over time.

A.7 Summary

The takeaway is methodological: the environment studied here is heterogeneous, economically nonstationary, and rich enough to justify treating dependence as time-varying. The descriptive evidence backs the modelling strategy of the main text.

B Appendix B: Supplementary Forecasting Results

This appendix gathers the extra forecasting diagnostics and model-comparison plots that complement the main discussion. It leans on figures rather than tables: changes in model behaviour come through far more clearly in a picture than in a very wide table.

B.1 Model-comparison figures

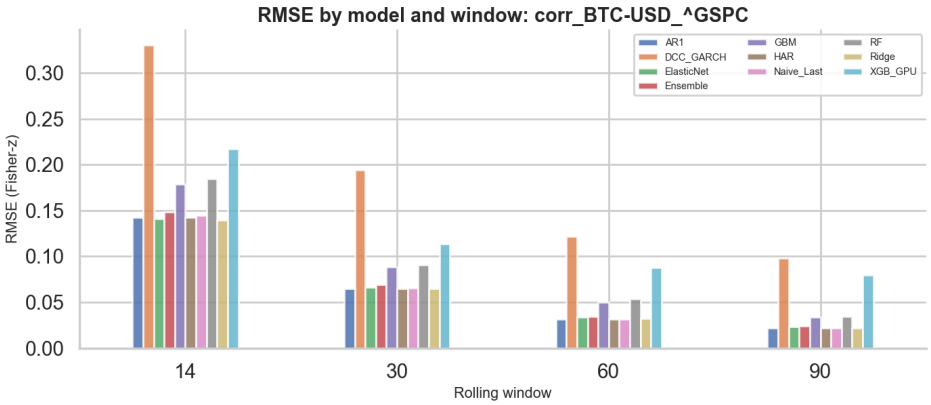


Figure 12: RMSE by model and window for the representative Bitcoin–S&P 500 pair.

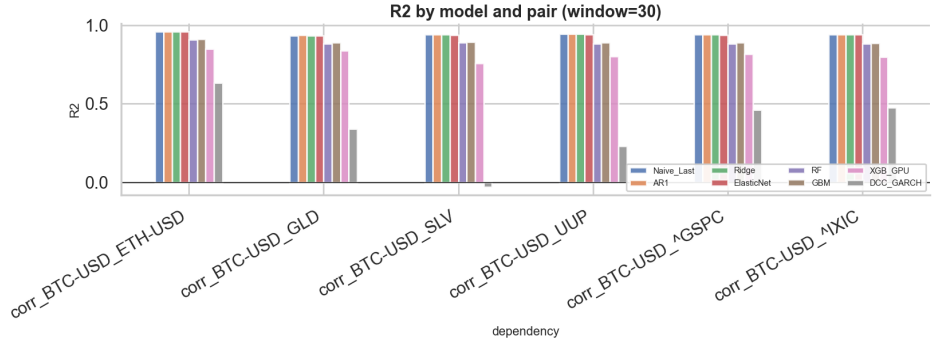


Figure 13: Model R^2 comparison for the 30-day window.

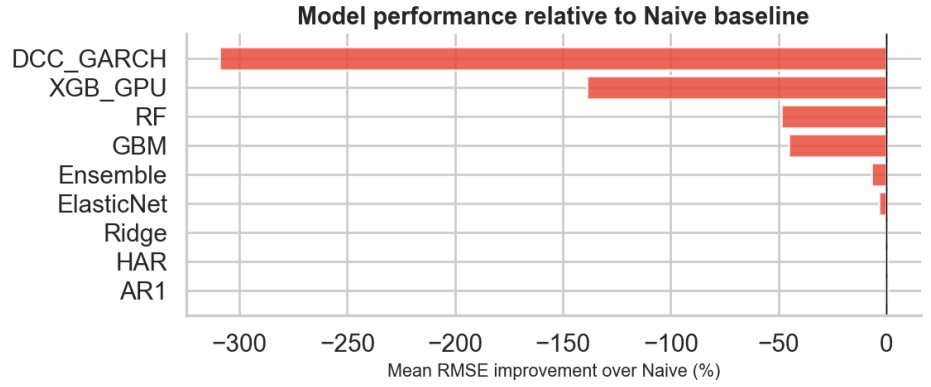


Figure 14: Improvement relative to the naive baseline.

B.2 Hyperparameter and feature diagnostics

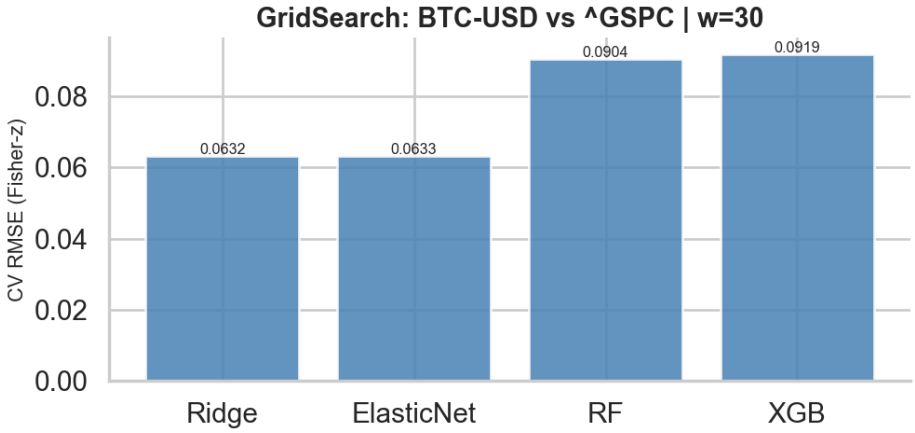


Figure 15: Grid-search cross-validation RMSE for the representative Bitcoin–S&P 500 forecasting task.

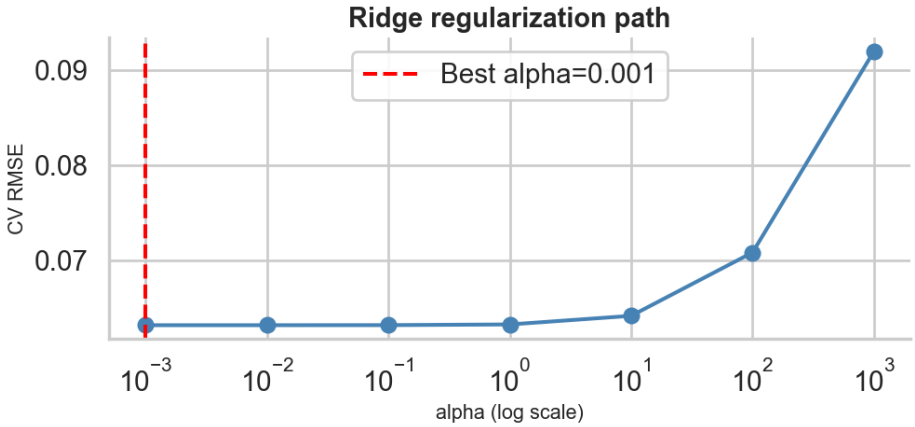


Figure 16: Regularization path for the Ridge-based forecasting specification.

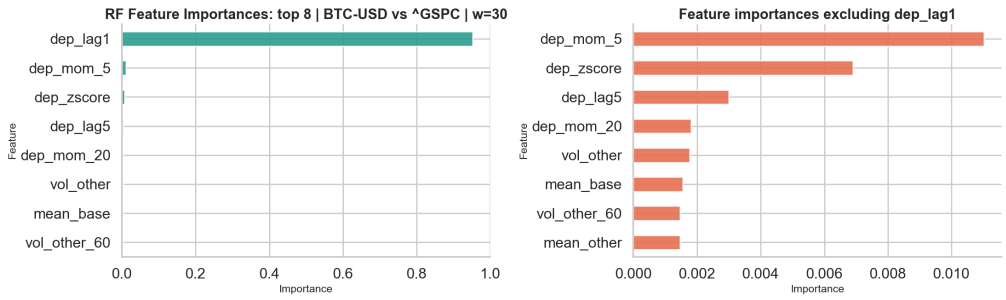


Figure 17: Random Forest feature importances for the Bitcoin–S&P 500 pair at the 30-day window.

B.3 Diagnostic plots for the representative equity pair

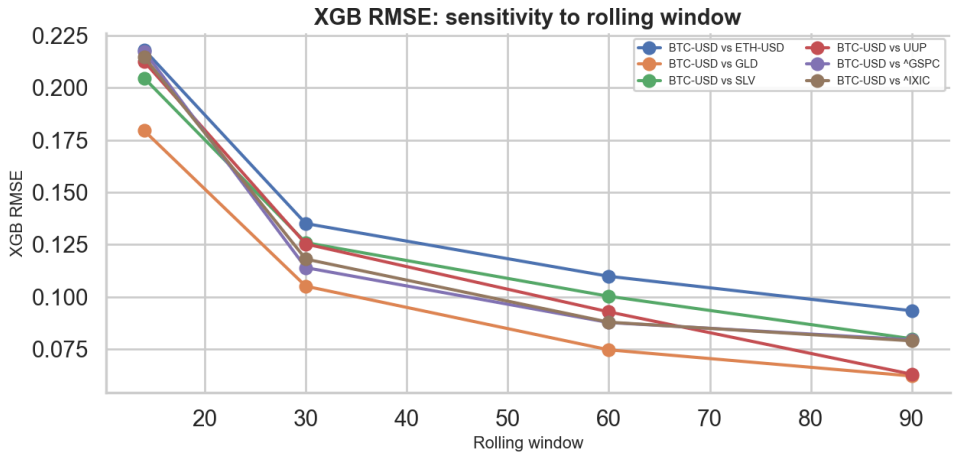


Figure 18: Window sensitivity of the XGBoost-based forecasting layer.

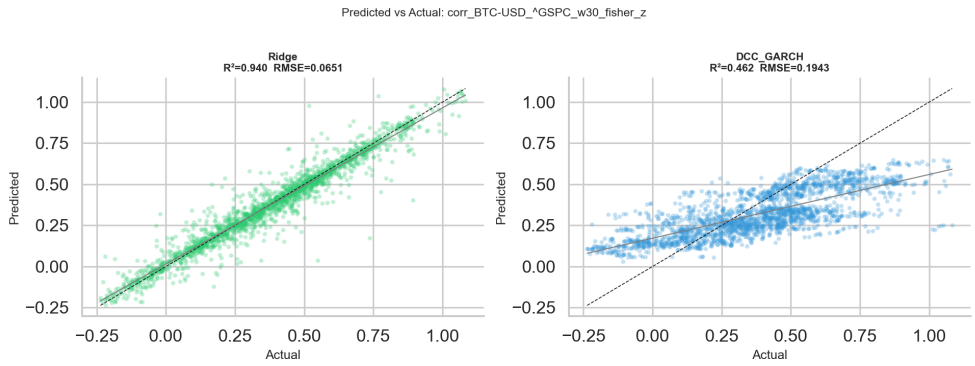


Figure 19: Forecast scatter diagnostic for the Bitcoin–S&P 500 dependency experiment.

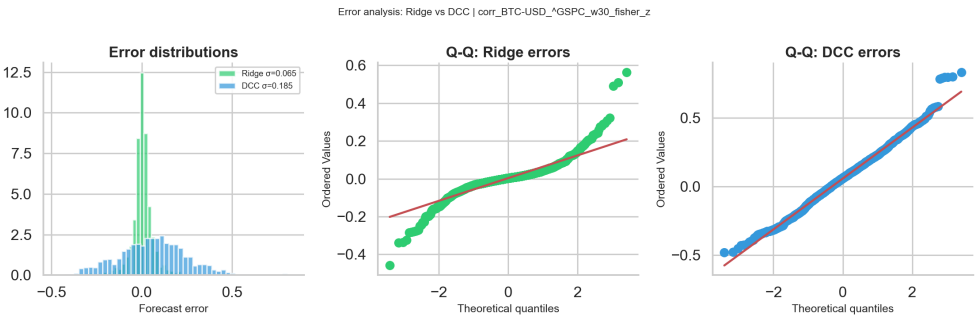


Figure 20: Forecast error distribution for the representative Bitcoin–S&P 500 dependency experiment.

B.4 Sensitivity by asset pair

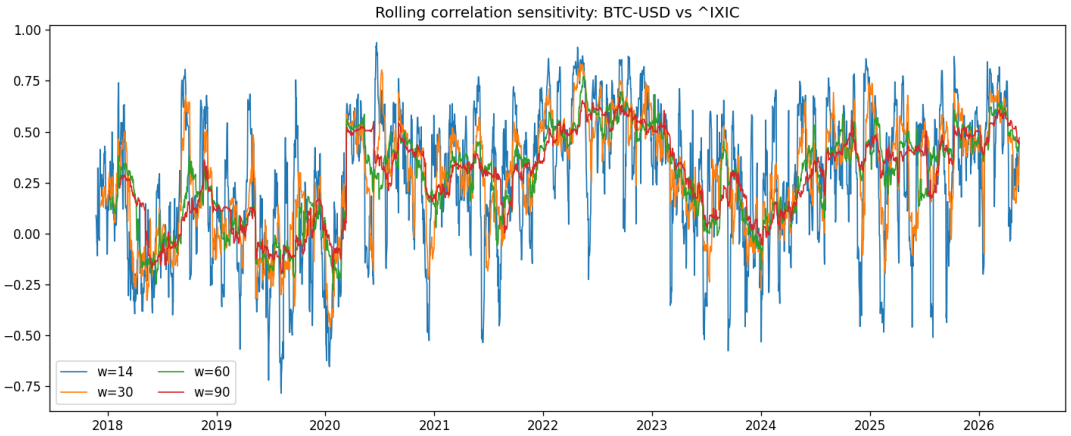


Figure 21: Sensitivity analysis for the Bitcoin–NASDAQ pair.

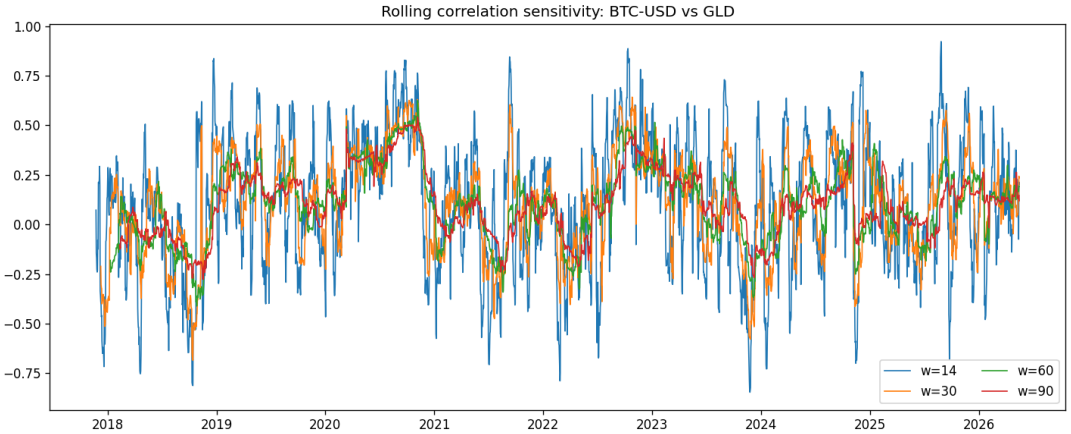


Figure 22: Sensitivity analysis for the Bitcoin–gold pair.

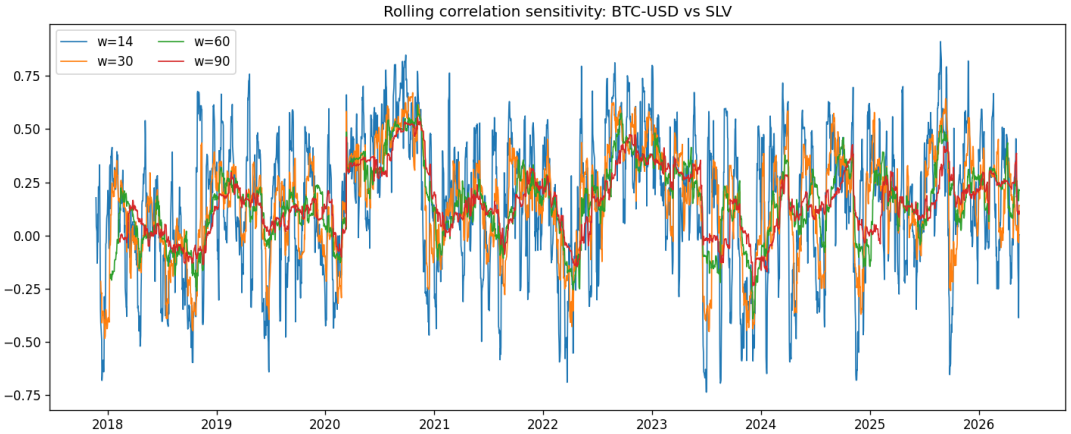


Figure 23: Sensitivity analysis for the Bitcoin–silver pair.

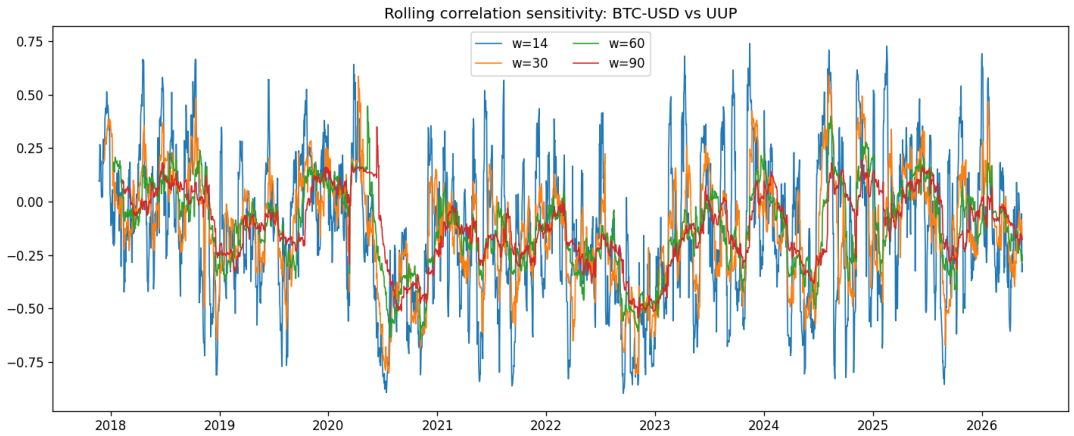


Figure 24: Sensitivity analysis for the Bitcoin–UUP pair.

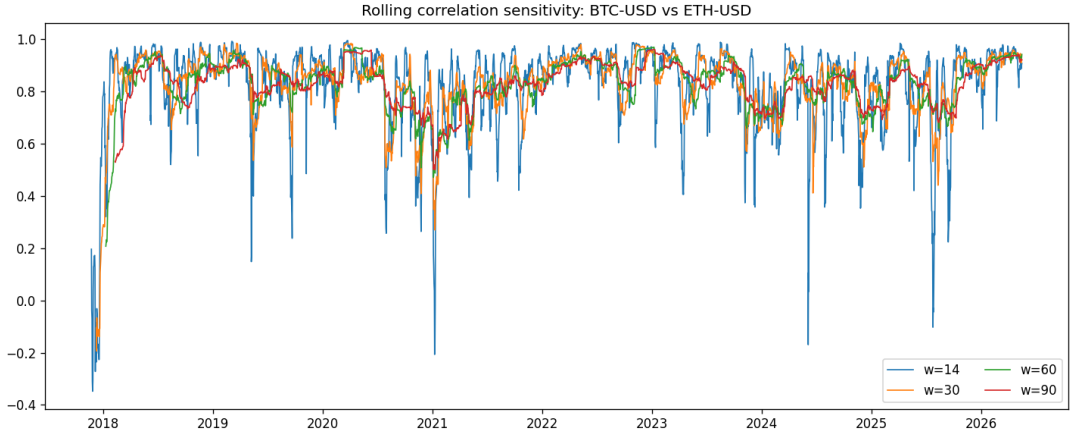


Figure 25: Sensitivity analysis for the Bitcoin–Ethereum pair.

B.5 Interpretation

Taken together, these figures reinforce the main text: forecast quality depends heavily on the rolling window, the target stays strongly persistence-driven, and the differences between model families read more clearly off a comparative plot than off any single ranking number.

C Appendix C: Supplementary Signal-Layer Results

This appendix collects the supporting material for the investor warning layer. A representative signal output is shown below, and the per-pair, per-window summary is given in Table 7. The full set of signal figures for all pairs and windows is produced by the pipeline and ships with the repository.

C.1 Representative signal output

Figure 26 shows the signal for the Bitcoin–NASDAQ pair at the 30-day window, the weakest of the equity pairs. It contrasts with the stronger Bitcoin–S&P 500 output shown in the main text (Figure 5).

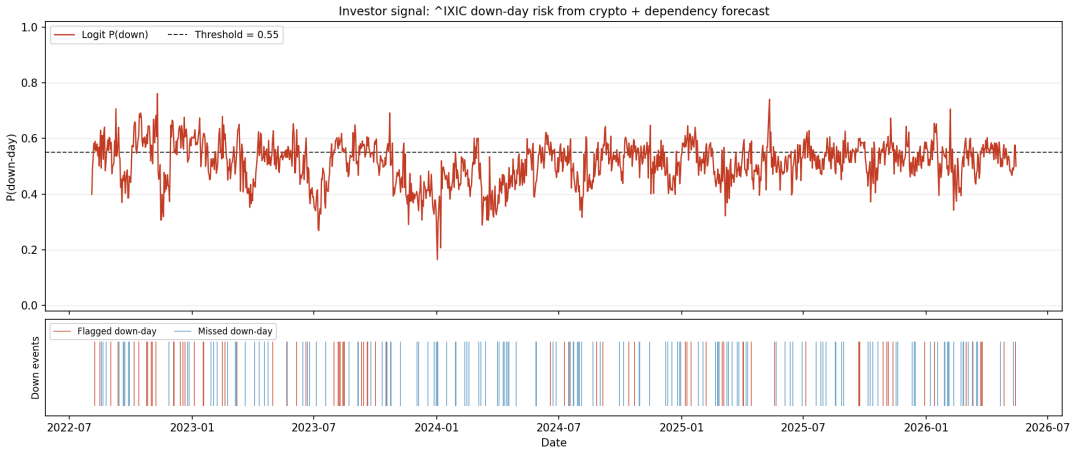


Figure 26: Signal-layer output for the Bitcoin–NASDAQ relation at the 30-day window. The upper panel shows the predicted stress probability and the decision threshold; the lower panel marks flagged and missed stress events.

C.2 Cross-market warning interpretation

The per-pair results show that the warning layer behaves very differently from market to market. The S&P 500 produces the clearest warning episodes; NASDAQ is markedly weaker, dipping below chance at the longer windows, probably because its sector concentration adds idiosyncratic noise that the crypto channel cannot see. The metal and dollar targets are noisier but not uniformly useless. The signal should therefore be applied selectively rather than across the asset universe.

C.3 Summary table

Table 7: Balanced accuracy of the Logit signal layer by asset pair and rolling-window length. Family-level averages across all pairs and windows are reported in Table 4; this breakdown shows where the signal is informative.

Asset pair	w=14	w=30	w=60	w=90	Mean
BTC–S&P 500	0.5169	0.5308	0.5212	0.5264	0.5238
BTC–NASDAQ	0.5190	0.5016	0.4865	0.4917	0.4997
BTC–Gold	0.5053	0.5041	0.5078	0.5202	0.5093
BTC–Silver	0.4938	0.5187	0.5053	0.5189	0.5092
BTC–USD index	0.5071	0.5197	0.5131	0.5204	0.5151

This breakdown supports the main-text reading. The Bitcoin–S&P 500 pair is the strongest and most consistent, peaking at 0.5308 at the 30-day window. Bitcoin–NASDAQ is the weakest, averaging 0.4997 and falling below the 0.50 chance level at the 60- and 90-day windows. The precious-metal and dollar pairs are positive on average but remain close to chance. The signal is therefore best applied selectively to broad-equity pairs rather than uniformly across the asset universe.

D Appendix D: Reproducibility and Code Structure

D.1 Project organization

The computational side of the thesis is organised as a reproducible research project rather than a collection of ad hoc scripts. The main directories are laid out as follows:

Listing 5: Top-level project directory structure.

```
master-thesis/
|-- main.py                  # main pipeline entry point
|-- run_all.py               # full automation: pipeline + notebooks +
    ↪ LaTeX
|-- config.yaml              # experiment configuration
|-- requirements.txt         # Python dependencies
|-- thesis_app/              # core Python modules
|   |-- pipeline.py          # data, features, walk-forward, metrics
|   |-- dcc.py               # DCC-GARCH(1,1) likelihood + GARCH recursion
|   |-- dcc_walk.py          # leakage-safe walk-forward wrapper (uses dcc.
    ↪ py)
|   |-- signal_layer.py      # investor stress-warning layer
|   |-- notebook_helpers.py  # shared plotting and styling
|   |-- data_quality.py      # missing-value and outlier diagnostics
|   `-- regime_analysis.py   # historical regime catalogue
|-- notebooks/               # eight Jupyter notebooks (01-08)
|-- data/raw/                # cached downloaded price data
|-- data/processed/          # derived returns
|-- outputs/results/         # CSV summaries of experiments
|-- outputs/tables/          # LaTeX-exported tables
|-- outputs/figures/         # all generated figures
`-- thesis/                  # this document
```

D.2 Main implementation modules

The following modules constitute the core of the forecasting system:

- `pipeline.py`: orchestrates the full dependency-forecasting and signal-generation workflow.
- `dcc.py`: implements the core DCC-GARCH(1,1) log-likelihood and the one-step GARCH variance recursion used by the benchmark.
- `dcc_walk.py`: contains the leakage-safe walk-forward DCC-GARCH implementation, built on the routines in `dcc.py`.
- `signal_layer.py`: defines the investor warning layer and associated classification metrics.
- `notebook_helpers.py`: shared plotting helpers, consistent styling, and analysis utilities used across all notebooks.
- `data_quality.py`: performs missing-value, outlier, and coverage diagnostics on the raw dataset.

- `regime_analysis.py`: implements the historical regime catalogue and regime-conditional statistics.

Splitting the work this way pays off twice over: it makes the workflow easier to audit, and it lets any one module be tested or improved without touching the rest.

D.3 Analysis notebooks

Eight Jupyter notebooks in `notebooks/` supplement the main pipeline with interactive diagnostics, publication-quality figures, and robustness checks. Each one is self-contained and runs either from the command line or interactively in Jupyter.

1. **`01_EDA_Dataset.ipynb`** — Exploratory data analysis. Produces normalized price trajectories, rolling volatility profiles, a pairwise correlation heatmap, rolling pairwise correlation time series, and the Fisher-transformation comparison plot. These figures appear in Appendix A.

2. **`02_GridSearch.ipynb`** — Hyperparameter search diagnostics. Reports time-series cross-validation (expanding-window `TimeSeriesSplit`) RMSE surfaces for the representative Bitcoin–S&P 500 experiment, regularization-path plots for Ridge and ElasticNet, and Random Forest feature-importance charts. Results support the model-selection discussion in Chapter 3.

3. **`03_Model_Comparison.ipynb`** — Model-ranking summary. Generates the average RMSE comparison bar chart, the R^2 comparison across window lengths, and the improvement-over-naive baseline figure across all asset pairs. These figures provide a visual complement to Table 3 in Chapter 4.

4. **`04_DM_Tests_Visuals.ipynb`** — Diebold–Mariano test visualization. Produces the DM heatmap comparing the best machine-learning model against DCC-GARCH (Figure 3) and the representative out-of-sample forecast panel for the Bitcoin–S&P 500 pair (Figure 2).

5. **`05_XGB_vs_DCC.ipynb`** — Error-profile diagnostics. Generates the rolling RMSE time series (Figure 4), forecast-error scatter plots, and error-distribution comparison histograms for the representative experiment.

6. **`06_Regime_Analysis.ipynb`** — Regime-conditional analysis. Produces the regime catalogue statistics table, regime-conditional RMSE bar charts, and data-quality summary figures. Provides the empirical basis for the regime-sensitivity discussion in Chapter 4.

7. **`07_Robustness_Checks.ipynb`** — Robustness and sensitivity analyses. Contains refit-frequency sensitivity sweeps, stress-threshold sensitivity curves, bootstrap confidence intervals for key metrics, signal-layer diagnostics, and the cross-asset Diebold–Mariano test comparing equity against non-equity pair performance.

8. **`08_Market_Events_Showcase.ipynb`** — Landmark market-event case studies. For each of 11 key episodes between 2020 and 2026 (COVID crash, Luna collapse, FTX bankruptcy, SVB crisis, Bitcoin ETF approval, Trump election rally, Liberation Day tariff shock, and others), the notebook produces a four-panel diagnostic figure showing normalized price action, daily return distributions, rolling correlation dynamics across window lengths, and a pre/post correlation shift

bar chart. A correlation-regime heatmap summarises all events jointly, and a scatter-plot grid illustrates how co-movement between Bitcoin and the S&P 500 varies by event type. All figures are saved to `outputs/figures/event_*.png` and `outputs/figures/market_events_*.png`; a summary CSV is written to `outputs/results/market_events_summary.csv`. Embedded HTML cells include direct links to the original news sources for each event. This notebook runs last so that it can draw on the full set of pipeline outputs and serves as a narrative complement to the quantitative results of Chapters 4 and 5.

All notebooks use a shared ROOT-detection routine that locates the project root by searching upward for `config.yaml`. This ensures correct path resolution whether a notebook is launched interactively from within the `notebooks/` directory or executed programmatically via `run_all.py` from the project root.

D.4 Execution flow

The full pipeline is launched through a single entry point that coordinates data acquisition, model training, notebook execution, and document compilation:

Listing 6: Automated full-pipeline runner (`run_all.py`), simplified for readability.

```
import argparse, subprocess, sys, os

BASE = os.path.dirname(os.path.abspath(__file__))
PYTHON = sys.executable

# Step 1: main pipeline (data, models, tables, figures)
subprocess.run([PYTHON, "main.py"], cwd=BASE, check=True)

# Step 2: execute each notebook with cwd set to project root
for nb in ["01_EDA_Dataset.ipynb", ..., "08_Market_Events_Showcase.ipynb"]:
    subprocess.run(
        [PYTHON, "-m", "jupyter", "nbconvert",
         "--to", "notebook", "--execute", "--inplace",
         f"--ExecutePreprocessor.cwd={BASE}",
         os.path.join(BASE, "notebooks", nb)],
        cwd=BASE
    )

# Step 3: run automated test suite (81 unit and integration tests)
subprocess.run(
    [PYTHON, "-m", "pytest", "tests/", "-v",
     "--junit-xml=outputs/results/test_results.xml"],
    cwd=BASE
)

# Step 4: compile the thesis PDF (test report table auto-included)
subprocess.run(
    ["latexmk", "-pdf", "-interaction=nonstopmode", "main.tex"],
    cwd=os.path.join(BASE, "thesis"), check=True
)
```

At a high level, the execution flow can be summarized as follows:

1. Load configuration and path settings.
2. Load or refresh raw price data from Yahoo Finance.
3. Compute synchronized log returns.
4. Generate rolling-correlation targets for each asset pair and window.
5. Build lagged and volatility-based feature matrices.
6. Fit all models in walk-forward mode and save predictions.
7. Compute metrics, DM tests, and export tables and figures.
8. Run the investor signal layer and export signal figures.
9. Execute eight Jupyter notebooks to produce supplementary diagnostics, including the market-event showcase (Appendix F).
10. Run the automated `pytest` suite (81 unit and integration tests) and generate the test-report table for Appendix E.
11. Compile the LaTeX thesis document.

The payoff is traceability: every figure and number in the thesis can be traced back to one consistent computational procedure.

D.5 Configuration

Experiment parameters are stored in `config.yaml` and are read once at pipeline startup. This makes it straightforward to change the date range, asset universe, or window set without touching the source code:

Listing 7: Excerpt from `config.yaml` showing key experiment parameters.

```
start_date: "2016-01-01"   # effective start: 2017-11-09 (ETH availability)
end_date:   "2026-05-17"   # frozen window; last trading day in data: 16
    ↪ May 2026

assets:
  crypto:      ["BTC-USD", "ETH-USD"]
  traditional: ["^GSPC", "^IXIC", "GLD", "SLV", "UUP"]

base_asset:          "BTC-USD"
rolling_windows:     [14, 30, 60, 90]
forecast_horizon:    1
use_fisher_transform: true
min_train_size:      800
refit_every:         20
xgb_device:          "cuda"
random_state:        42
```

D.6 Verification performed during the project cleanup

While tightening the project up, several implementation issues were fixed to bring the code to thesis standard.

- A leakage-safe walk-forward DCC benchmark was introduced to avoid full-sample look-ahead contamination.
- The metrics pipeline was made consistent so that all relevant models, including DCC, are evaluated through the same summary path.
- Feature scaling was added for the regularized linear models.
- The XGBoost block was made robust to GPU failure through CPU fallback.
- The investor signal layer was integrated into the main pipeline and exported to dedicated tables and figures.
- Notebook imports and notebook-specific plotting logic were stabilized for presentation and interpretation.

These fixes matter because a thesis is judged not only on its idea but on whether the implementation behind it is reliable, reproducible, and internally coherent.

D.7 How to rebuild the thesis artifacts

A practical benefit of the current project state is that the full workflow can be rerun from scratch with a single command from the project root:

Listing 8: Full rebuild from the project root directory.

```
python run_all.py
```

Alternatively, the three stages can be executed individually:

1. Run the main pipeline:
   `python main.py`
   This generates prediction CSVs, metrics, DM-test results, and the majority of figures.
2. Execute the notebooks in order:
   (a) `01_EDA_Dataset.ipynb` — dataset overview and EDA figures.
   (b) `02_GridSearch.ipynb` — cross-validated hyperparameter search.
   (c) `03_Model_Comparison.ipynb` — model-ranking table and heatmaps.
   (d) `04_DM_Tests_Visuals.ipynb` — DM-test heatmap and key forecast figure.
   (e) `05_XGB_vs_DCC.ipynb` — rolling-RMSE, scatter, and error-distribution figures.
   (f) `06_Regime_Analysis.ipynb` — regime catalogue and data-quality diagnostics.
   (g) `07_Robustness_Checks.ipynb` — refit sensitivity, stress threshold sensitivity, bootstrap confidence intervals, signal diagnostics, and cross-asset DM test.

(h) `08_Market_Events_Showcase.ipynb` — landmark market-event case studies (Appendix F).

3. Run the automated test suite and generate the appendix table:

```
python -m pytest tests/ -v --junit-xml=outputs/results/test_results.
    ↪ xml
```

4. Rebuild the PDF document from the `thesis/` directory with the same command the runner uses:

```
latexmk -pdf -interaction=nonstopmode main.tex
```

   If `latexmk` is not installed, the equivalent manual sequence is `pdflatex main.tex`, then `biber main`, then `pdflatex main.tex` run twice.

Steps 1 and 2 must be completed before step 4 because several figures used in the thesis are produced by the notebooks. Step 3 must be completed before step 4 so that the test-report table (Appendix E) is up to date. This makes the written work maintainable even if the underlying market data are refreshed.

D.8 Final reproducibility note

Reproducibility here is not just a technical nicety. In financial forecasting, where accidental leakage and selective reporting are ever-present temptations, a transparent, rerunnable codebase is part of what makes the empirical conclusions believable.

E Appendix E: Additional Statistical Tests

This appendix runs three further hypothesis tests that probe the robustness of the empirical results: a Mincer-Zarnowitz efficiency test, residual diagnostics (Ljung-Box and ARCH LM), and a Harvey-Leybourne-Newbold small-sample correction of the Diebold-Mariano statistic.

E.1 Mincer-Zarnowitz Forecast Efficiency Test

The Mincer-Zarnowitz (MZ) test [26] checks whether forecasts are unbiased and informationally efficient. For each model the auxiliary regression

$$y_t = \alpha + \beta\, \hat{y}_t + \varepsilon_t \tag{10}$$

is estimated by OLS and the joint null hypothesis H_0: $\alpha = 0,\ \beta = 1$ is tested with an F-statistic. Rejection ($p < 0.05$) indicates that the forecast is either biased ($\alpha \neq 0$) or fails to make full use of available information ($\beta \neq 1$).

Table 8 summarises the percentage of the 24 dependency–window experiments in which each model passes the MZ test, together with its average α and β coefficients.

Table 8: Mincer-Zarnowitz forecast efficiency: percentage of 24 experiments passing the joint F-test ($p \geq 0.05$) and average coefficient estimates.

Model	Efficient (%)	$\bar{\alpha}$	$\bar{\beta}$	$\bar{F}$
AR1	70.8	+0.0006	0.9976	2.0
HAR	70.8	+0.0006	0.9974	2.4
Ridge	45.8	−0.0015	0.9992	4.1
RF	16.7	−0.0019	1.0140	26.2
Naive_Last	8.3	+0.0097	0.9706	16.3
GBM	8.3	−0.0046	1.0226	35.0
ElasticNet	4.2	−0.0056	1.0175	20.9
Ensemble	0.0	−0.0077	1.0252	30.1
XGB_GPU	0.0	−0.0252	1.0979	202.2
DCC_GARCH	0.0	−0.0881	1.4433	237.3

AR1 and HAR pass the efficiency test at the same rate (70.8% of experiments), confirming that both models make near-optimal use of the available information. Ridge is the most efficient among the regularised ML models at 45.8%. DCC-GARCH never passes: its average $\bar{\beta} = 1.44$ means that realized correlations swing markedly more than its forecasts do. In the regression $y = \alpha + \beta\hat{y}$, a slope above one signals forecasts that are too *compressed* — they systematically under-react to the size of the movements rather than overstating them. This is precisely the over-smoothing of DCC-GARCH seen throughout the main results.

Table 9 provides the full detail for the representative pair BTC-USD vs S&P 500 at $w = 30$.

Table 9: Mincer-Zarnowitz test: BTC-USD vs S&P 500, $w = 30$. F-statistic tests H_0: $\alpha = 0, \beta = 1$; p_{MZ} is the associated p-value.

Model	$\hat{\alpha}$	$\hat{\beta}$	F	p_{MZ}	Efficient
Ridge	+0.0071	0.9903	4.76	0.009	No
AR1	+0.0071	0.9909	5.07	0.006	No
HAR	+0.0074	0.9913	5.77	0.003	No
Naive_Last	+0.0114	0.9695	16.74	< 0.001	No
ElasticNet	+0.0065	1.0048	17.18	< 0.001	No
Ensemble	+0.0086	1.0113	39.91	< 0.001	No
RF	+0.0221	0.9951	57.04	< 0.001	No
GBM	+0.0226	0.9945	62.30	< 0.001	No
DCC_GARCH	−0.0886	1.4616	261.03	< 0.001	No
XGB_GPU	+0.0436	1.0316	318.18	< 0.001	No

At this particular pair and window all models are technically rejected, but the performance gradient is steep. Ridge ($F = 4.76$), AR1 ($F = 5.07$), and HAR ($F = 5.77$) are rejected at the boundary of significance and have coefficients very close to $(0, 1)$, indicating near-efficient use of information. The DCC-GARCH statistic (261.0) and XGBoost statistic (318.2) stand apart as the most severely misspecified, confirming the main-text findings.

E.2 Residual Diagnostics: Ljung-Box and ARCH LM Tests

Two classical residual tests are applied to the OOS forecast errors $e_t = y_t - \hat{y}_t$ for every model × dependency × window combination (240 cases in total).

Ljung-Box Q-test (10 lags) [27]: tests H_0: no serial autocorrelation. Rejection indicates that the model has not fully captured the temporal structure of the target series.

ARCH LM test (5 lags) [28]: regresses e_t^2 on its own lags and tests H_0: no conditional heteroscedasticity. Rejection signals remaining volatility clustering in the residuals.

Table 10 reports, for each model, the percentage of experiments in which the residuals pass each test at the 5% level.

Table 10: Residual diagnostics: percentage of 24 experiments passing the Ljung-Box autocorrelation test and the ARCH LM heteroscedasticity test at $\alpha = 0.05$.

Model	No autocorr (%)	No ARCH (%)
Naive_Last	50.0	58.3
AR1	41.7	62.5
HAR	41.7	58.3
Ridge	20.8	54.2
ElasticNet	0.0	37.5
Ensemble	0.0	12.5
RF	0.0	0.0
GBM	0.0	0.0
XGB_GPU	0.0	0.0
DCC_GARCH	0.0	0.0

The autocorrelation results follow a clear gradient. Naive_Last and the autoregressive models (AR1, HAR) partly capture the serial structure, passing the Ljung-Box test in 42–50% of experiments. Ridge passes in 21% of cases, reflecting the autoregressive signal it picks up from the lag features. ElasticNet is an odd case: it fails every autocorrelation test (0%) yet keeps a meaningful ARCH pass rate (38%), so it removes some temporal dependence but not all. The tree-based methods (RF, GBM, XGBoost) and DCC-GARCH leave strongly significant autocorrelation and heteroscedasticity in every single experiment: they never encode the target's autoregressive structure, so that structure leaks into the residuals, which remain far from white noise. The enormous Q-statistics for the tree ensembles (e.g. $Q > 1{,}000$ for XGB_GPU on most pairs) drive the point home — these forecasters simply do not carry the AR structure forward from training.

E.3 Harvey-Leybourne-Newbold Corrected Diebold-Mariano Test

The standard Diebold-Mariano statistic uses the asymptotic standard-normal distribution, which tends to over-reject in finite samples. Harvey, Leybourne & Newbold [29] propose multiplying the DM statistic by the correction factor

$$k = \sqrt{\frac{n + 1 - 2h + h(h-1)/n}{n}}, \tag{11}$$

where n is the number of forecast observations and h the forecast horizon, and comparing the result to a $t(n-1)$ distribution. For $h = 1$ and $n > 1{,}000$ the factor $k \approx 1.000$, so the practical effect on p-values is negligible.

Applied to all 336 DM comparisons (168 ML models vs DCC-GARCH and 168 vs Naive_Last, Table 11) the HLN correction produces zero sign changes in significance at the 5% level (average correction factor $k = 1.000226$). All conclusions regarding the relative ranking of models, and in particular the finding that every ML model significantly outperforms DCC-GARCH, are therefore robust to the small-sample correction.

Table 11: HLN-corrected DM test: ML models vs DCC-GARCH benchmark, BTC-USD vs S&P 500, $w = 30$. DM: original statistic (normal); DM_{HLN}: corrected statistic (t_{n-1}). All models beat DCC-GARCH at $p < 0.001$ under both tests.

Model	DM	p_{DM}	DM_{HLN}	p_{HLN}	Sig.
Ridge	26.86	< 0.001	26.86	< 0.001	***
HAR	26.83	< 0.001	26.83	< 0.001	***
ElasticNet	26.97	< 0.001	26.97	< 0.001	***
Ensemble	27.05	< 0.001	27.04	< 0.001	***
RF	26.44	< 0.001	26.43	< 0.001	***
GBM	26.16	< 0.001	26.16	< 0.001	***
XGB_GPU	24.29	< 0.001	24.28	< 0.001	***

Two aspects of the comparison set in Table 11 should be made explicit. First, the seven *fitted* models are tested against two benchmarks: the DCC-GARCH econometric model and the naive last-value baseline. AR(1) is treated as one of the persistence baselines rather than as a test model, so it does not appear as a separate row; its position relative to the fitted models is established instead by the average-RMSE ranking of Table 3, in which AR(1), HAR, and Ridge are statistically indistinguishable. Second, the Diebold–Mariano test assumes non-nested forecasts, whereas every fitted model here includes the lagged target among its predictors and therefore nests the persistence baselines. The statistics against Naive_Last, and by extension any test against AR(1), accordingly carry only descriptive weight; the conclusions rest on the comparison the test is designed for, namely the fitted models against the non-nested DCC-GARCH benchmark.

E.4 Automated Validation Suite

In addition to the statistical tests above, the full project ships with an automated `pytest` suite of **81 unit and integration tests** organised into ten test classes. These are predominantly consistency and regression checks: they verify that a fresh run reproduces the published outputs — file presence, value ranges, sign conventions, and key relationships such as the average ranking of AR(1) against DCC-GARCH — rather than proving the methods correct in an absolute sense. The suite is executed automatically as part of the reproducibility runner (`python run_all.py`) between the notebook step and the final LaTeX compilation, so every rebuilt PDF contains a fresh test report.

The test classes cover the following aspects of the pipeline:

- **Dataset Integrity** — raw price and return files exist, have correct tickers, positive prices, sorted and non-duplicate dates, and extend to at least 2026.
- **Data Quality** — missing-rate thresholds, zero-price detection, reasonable annualised volatility, pairwise coverage overlap.
- **Feature Engineering** — Fisher-z round-trip identity, rolling-correlation range, no look-ahead in prediction CSVs.
- **Model Metrics** — RMSE > 0, $R^2 \leq 1$, MAE $\leq$ RMSE, all windows and expected models present, AR1 beats DCC-GARCH on average.

- **DM Tests** — finite DM statistics, p-values in $[0, 1]$, correct sign convention for Ridge vs DCC-GARCH.
- **Signal Metrics** — balanced accuracy and F1 in $[0, 1]$, AUC above random, exit rate within a sensible range.
- **Walk-Forward** — correct number of prediction CSVs, non-trivial predictions, index alignment with returns.
- **Output Files** — all expected CSV, JSON, PNG, and `.tex` artefacts are present after a full pipeline run.
- **Bootstrap CI** — lower $\leq$ mean $\leq$ upper for RMSE and R^2; CI width below 0.10.
- **Sensitivity** — all five `refit_every` values present; RMSE in a physically plausible range.

Table 12 summarises the suite by test class — the number of tests, the pass rate, and the total execution time for each. The complete per-test log, with parametrised cases and individual timings, is stored in `outputs/results/test_results.xml` in the repository. All 81 tests pass on the present outputs, with none skipped.

Table 12: Automated validation suite, summarised by test class. All 81 tests pass on the present outputs, with none skipped. The complete per-test log, including parametrised cases and per-test timings, is written to `outputs/results/test_results.xml` in the repository.

Test class	Tests	Passed	Time (s)
`TestDatasetIntegrity`	13	13/13	0.08
`TestDataQuality`	9	9/9	0.04
`TestFeatureEngineering`	6	6/6	0.04
`TestModelMetrics`	12	12/12	0.02
`TestDMTests`	8	8/8	0.01
`TestSignalMetrics`	7	7/7	0.01
`TestWalkForward`	5	5/5	0.05
`TestOutputFiles`	14	14/14	0.00
`TestBootstrapCI`	3	3/3	0.00
`TestSensitivity`	4	4/4	0.00
Total	**81**	**81**	**0.26**

F Appendix F: Market-Event Case Studies

This appendix examines the landmark market events of 2020–2026 through the lens of the thesis's central variable: the rolling Pearson correlation between Bitcoin and conventional assets. A master timeline and a correlation regime map first summarise all eleven episodes jointly; detailed four-panel diagnostics are then provided for four representative events — the COVID crash, the FTX collapse, the SVB banking crisis, and the 2025 "Liberation Day" tariff shock — chosen to span the main regime types: a liquidity-driven co-crash, a crypto-idiosyncratic shock, a brief safe-haven decoupling, and a macro risk-off shock. The remaining seven episodes are marked on the master timeline and reproduced in full by notebook `08_Market_Events_Showcase.ipynb` in the repository. The purpose is to anchor the aggregate quantitative results of Chapter 4 in concrete historical episodes and to show how the regime-transition problem manifests in practice.

Master Timeline

Figure 27 presents the full sample period with all eleven events marked. The three panels show Bitcoin's normalised price trajectory (log scale), the S&P 500 and NASDAQ together with gold, and the 30-day rolling BTC–S&P 500 Pearson correlation. The event markers make visible how correlation spikes cluster around macro risk-off episodes while remaining subdued during crypto-specific events.

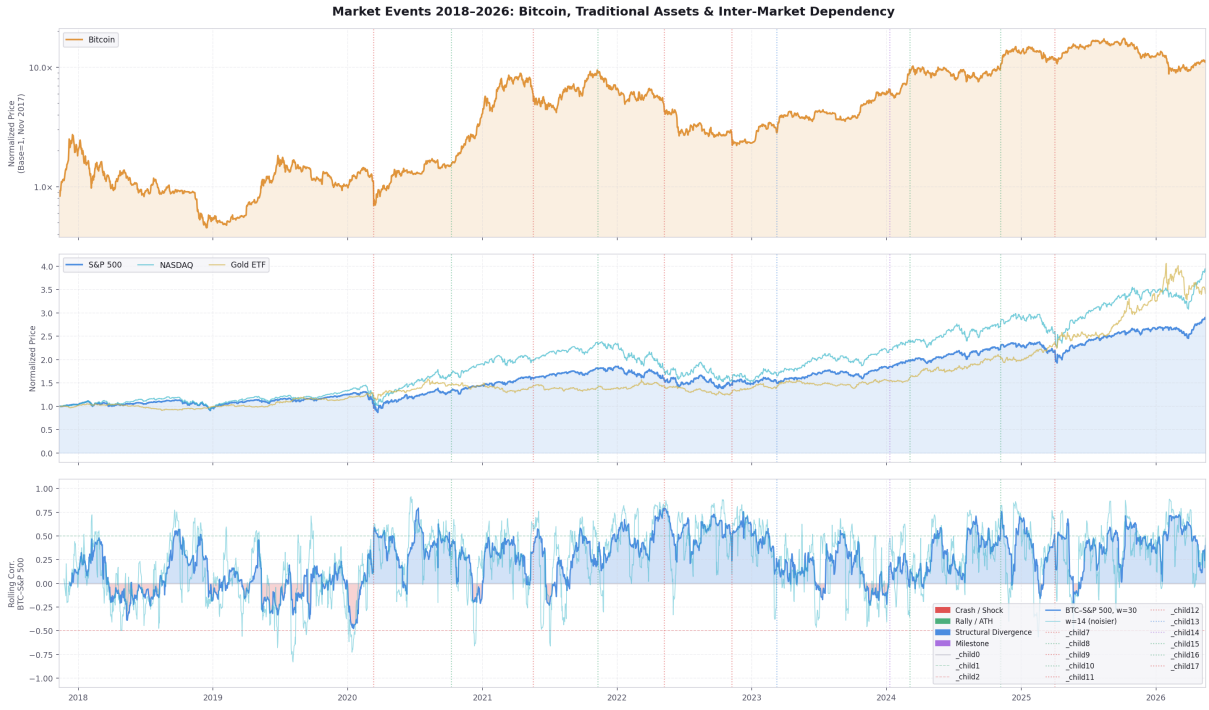


Figure 27: Master timeline 2018–2026: Bitcoin price, conventional-asset prices, and the 30-day rolling BTC–S&P 500 correlation, with all eleven landmark events marked. Red dotted lines = crash/shock events; green = rallies and all-time highs; blue = structural divergences; purple = institutional milestones.

Correlation Regime Map

Figure 28 summarises all events jointly. The left panel shows the 30-day average BTC correlation with each conventional asset *after* each event; the right panel shows the change in correlation ($\Delta\rho = \text{post} - \text{pre}$), isolating the regime shift attributable to the event itself.

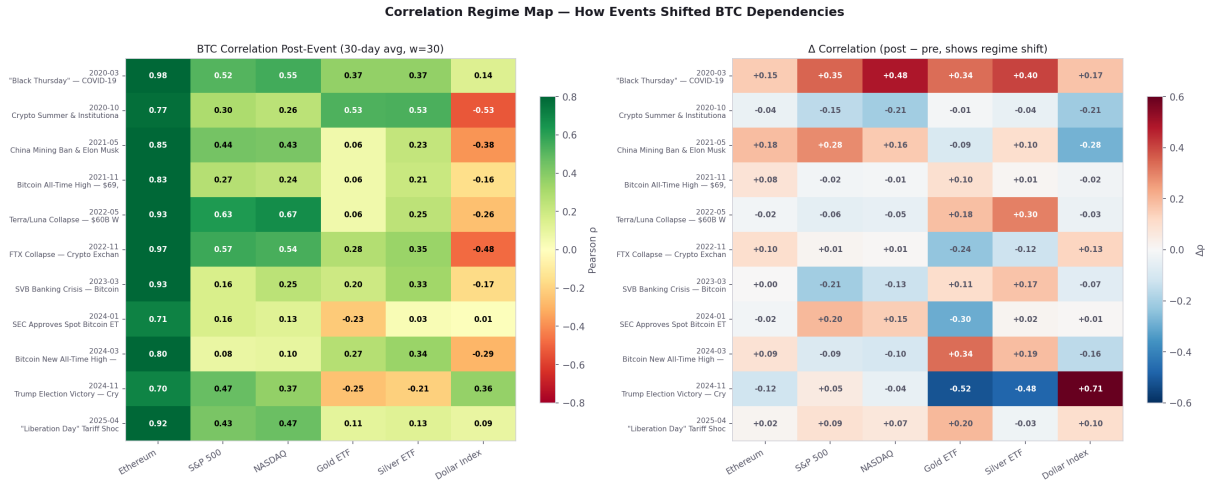


Figure 28: Correlation regime map. Left: post-event BTC correlation (30-day average) with each conventional asset. Right: change in correlation relative to the 30-day pre-event average. Red/blue = strong positive/negative shifts.

Individual Event Panels

The following figures provide a four-panel deep-dive for each of the four representative events: (top-left) normalised price trajectories for Bitcoin and the most relevant comparison assets; (top-right) daily log-returns as a bar chart; (bottom-left) 30-day and 14-day rolling BTC correlation with the primary comparison asset; (bottom-right) pre-/post-event correlation for all pairs.

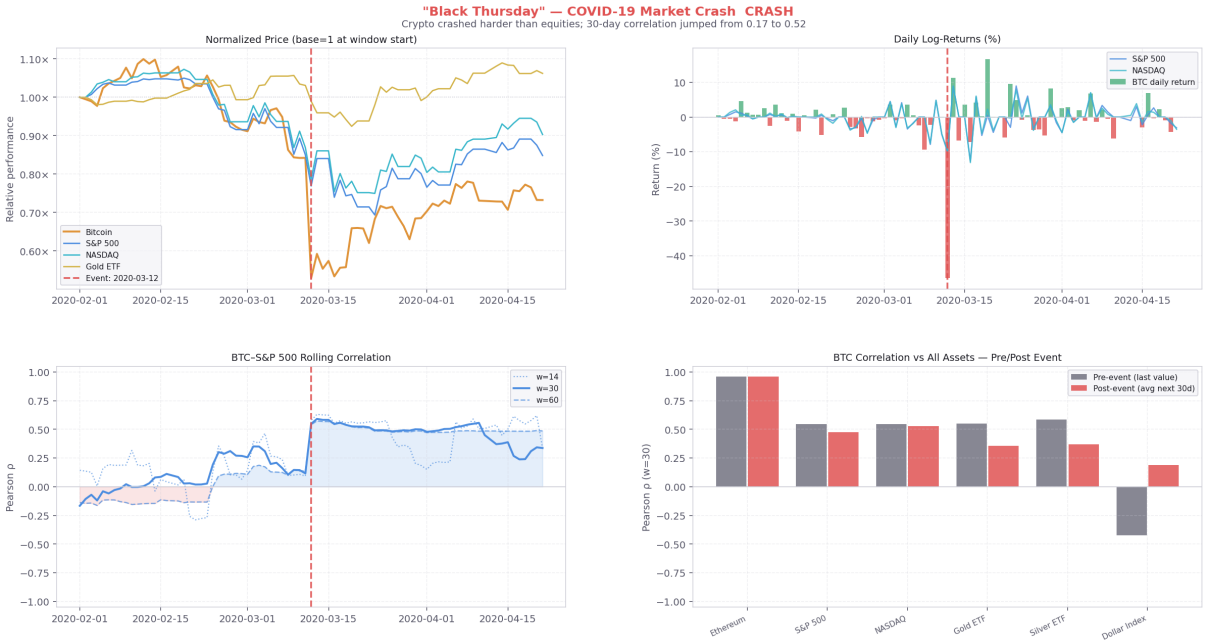


Figure 29: **Event 1 — COVID Black Thursday (12 March 2020).** Bitcoin fell about 37% in a single session, sharply outpacing the S&P 500's 10% drop. The 30-day BTC–S&P 500 correlation rose from near zero to about 0.52 — a liquidity-driven sell-off in which Bitcoin's diversification benefit temporarily disappeared. Source: Reuters

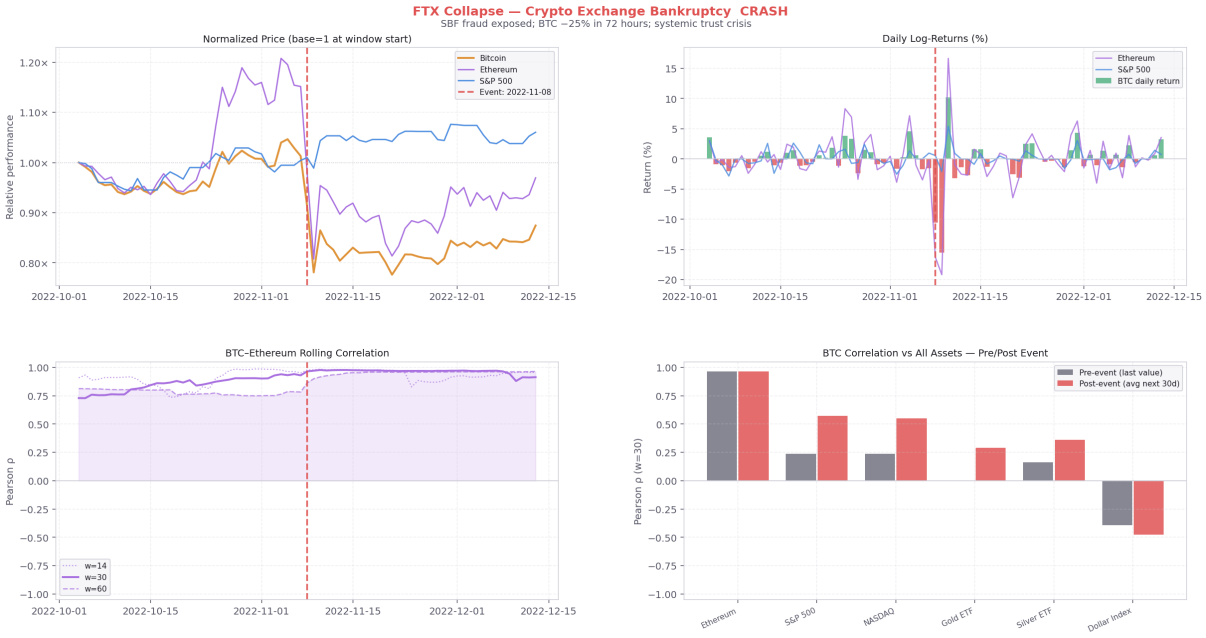


Figure 30: **Event 2 — FTX Collapse (8–11 November 2022).** SBF's exchange filed for bankruptcy; Bitcoin fell about 25% over the following two weeks. Paradoxically the S&P 500 *rose* that same week on a positive CPI print — producing a sharp short-term decoupling: the 14-day BTC–equity correlation fell from about 0.6 toward zero, though the 30-day measure barely moved (about 0.55). Source: BBC News

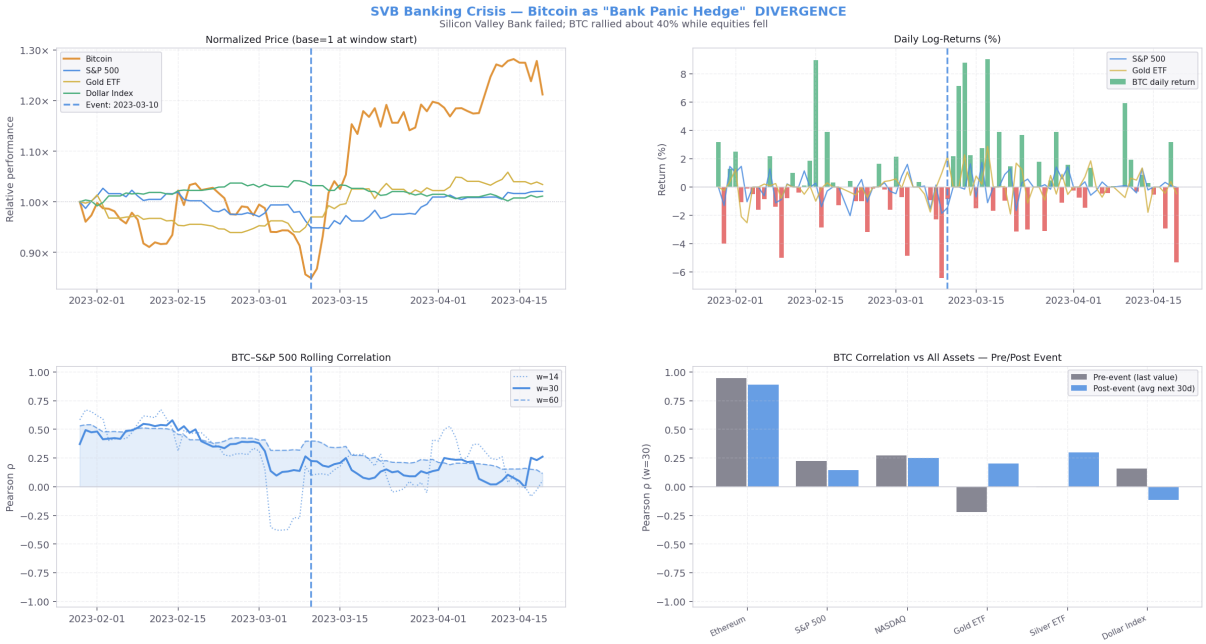


Figure 31: **Event 3 — SVB Banking Crisis (10 March 2023).** Silicon Valley Bank's failure triggered a Bitcoin rally of about 40% over the following week as investors sought alternatives to the banking system. Both Bitcoin and Gold rose (Gold about 8%) while equities fell — one of the rare episodes in the sample where the short-window correlation turned negative: the 14-day BTC–S&P 500 measure dipped to about −0.4, while the 30-day measure fell from 0.37 to 0.16. Source: The New York Times

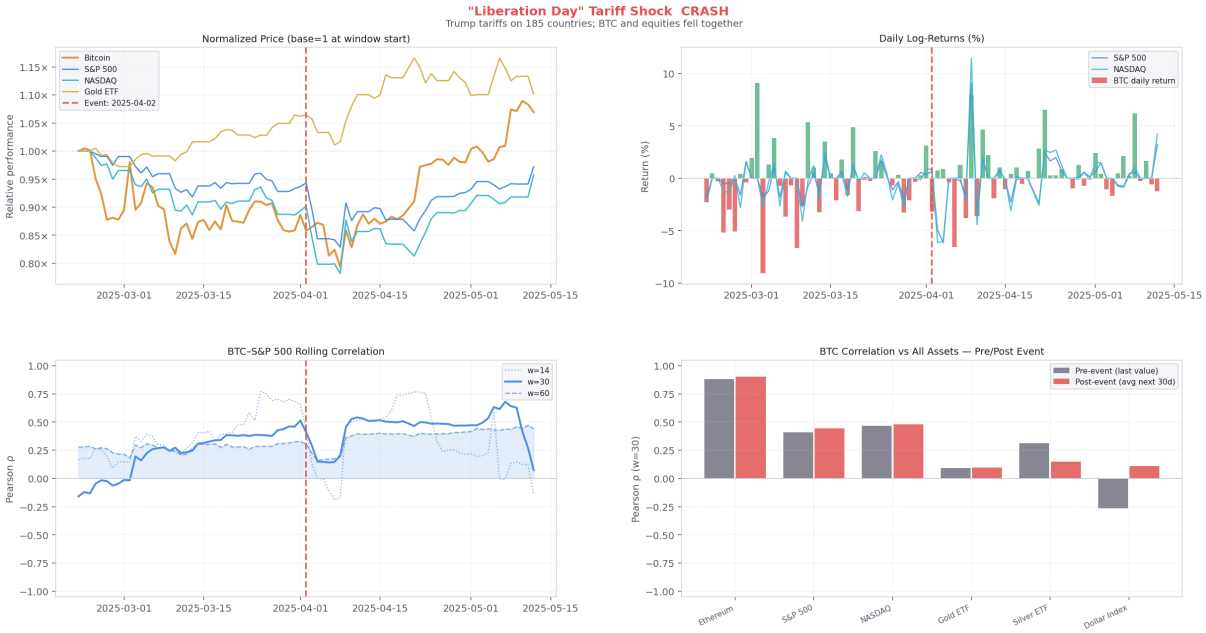


Figure 32: **Event 4 — "Liberation Day" Tariff Shock (2 April 2025).** Sweeping tariffs on 185 countries caused the S&P 500 to fall about 12% in four trading days — its fastest drop since March 2020. Bitcoin fell alongside equities over the same week, while Gold climbed to record highs above $3 000/oz. Although both risk assets sold off together — confirming that Bitcoin behaves as a risk asset, not a safe haven, during acute macro shocks — the 30-day rolling BTC–S&P 500 correlation showed no clear spike: it edged only from about 0.33 to 0.43, as the sharp partial recovery in the days that followed desynchronised the daily moves. The episode illustrates that even a major macro shock need not durably raise the rolling dependency. Source: Reuters

Interpretation

Across the full set of episodes — those detailed above together with the remainder marked on the master timeline — four conclusions complement the aggregate quantitative findings of Chapter 4:

1. **Macro risk-off events create the largest correlation spikes.** The COVID crash (2020) and the 2021–22 Fed tightening cycle pushed BTC–equity correlation to its highest sustained levels in the sample: the 30-day measure rose to about 0.52 in March 2020 and ran near 0.6–0.7 through the 2022 sell-off. These are precisely the episodes where rolling-RMSE peaks in Figure 4. By contrast, the 2025 "Liberation Day" shock was a co-crash in prices that left the rolling correlation little changed — a reminder that even severe macro shocks do not always raise measured dependency.

2. **Crypto-idiosyncratic shocks leave equity correlations largely unchanged.** The China mining ban, Luna collapse, and FTX bankruptcy each caused severe Bitcoin losses while equity markets barely moved, producing at most a brief, transitory swing in the rolling correlation rather than a sustained regime shift.

3. **The institutional era (post-2024 ETF approval) keeps correlations elevated.** The structural inflow of institutional capital via ETF wrappers means Bitcoin is now traded alongside growth equities in the same risk-on/off frameworks.

The 30-day BTC–NASDAQ correlation has remained elevated since the January 2024 ETF approval, averaging about 0.36 over 2024–2026, although it does not exceed the peak of about 0.58 reached during the 2022 monetary-tightening cycle, when both assets sold off together.

4. **Rare safe-haven episodes exist but are short-lived.** The SVB crisis produced a genuine negative BTC–equity correlation, but it lasted only two to three weeks before reverting. This transience explains why persistence-based models continue to dominate out-of-sample: the autoregressive signal is strong enough to absorb regime shifts after a short lag, even if it cannot anticipate them.

The full set of event-level figures, summary statistics, and news references for all eleven episodes is produced automatically by notebook `08_Market_Events_Showcase.ipynb`, which runs as the final step of `run_all.py`.

References

[1] François Longin and Bruno Solnik, „Extreme Correlation of International Equity Markets", in: *The Journal of Finance* 56.2 (2001), pp. 649–676.

[2] Satoshi Nakamoto, „Bitcoin: A peer-to-peer electronic cash system", in: *Decentralized Business Review* (2008).

[3] Elie Bouri et al., „On the hedge and safe haven properties of Bitcoin: Is it really more than a diversifier?", in: *Finance Research Letters* 20 (2017), pp. 192–198.

[4] Shaen Corbet et al., „Exploring the dynamic relationships between cryptocurrencies and other financial assets", in: *Economics Letters* 165 (2018), pp. 28–34.

[5] Qiang Ji et al., „Network causality structures among Bitcoin and other financial assets", in: *The Quarterly Review of Economics and Finance* 70 (2018), pp. 203–213.

[6] Ladislav Kristoufek, „What are Bitcoin's main drivers?", in: *PLOS ONE* 10.4 (2015), e0123923.

[7] Anne Haubo Dyhrberg, „Bitcoin, gold and the dollar – A GARCH volatility analysis", in: *Finance Research Letters* 16 (2016), pp. 85–92.

[8] Tony Klein, Hien Pham Thu, and Thomas Walther, „Bitcoin is not the new gold – A comparison of volatility, correlation, and portfolio performance", in: *International Review of Financial Analysis* 59 (2018), pp. 105–116.

[9] Andrew J. Patton, „Modelling asymmetric exchange rate dependence", in: *International Economic Review* 47.2 (2006), pp. 527–556.

[10] Robert F. Engle and Kevin Sheppard, „Theoretical and empirical properties of dynamic conditional correlation multivariate GARCH", in: *NBER Working Paper* 8554 (2001).

[11] Robert F. Engle, „Dynamic Conditional Correlation: A Simple Class of Multivariate Generalized Autoregressive Conditional Heteroskedasticity Models", in: *Journal of Business & Economic Statistics* 20.3 (2002), pp. 339–350.

[12] Shihao Gu, Bryan Kelly, and Dacheng Xiu, „Empirical asset pricing via machine learning", in: *The Review of Financial Studies* 33.5 (2020), pp. 2223–2273.

[13] Fulvio Corsi, „A simple approximate long-memory model of realized volatility", in: *Journal of Financial Econometrics* 7.2 (2009), pp. 174–196.

[14] Torben G. Andersen et al., „Modeling and forecasting realized volatility", in: *Econometrica* 71.2 (2003), pp. 579–625.

[15] Hui Zou and Trevor Hastie, „Regularization and Variable Selection via the Elastic Net", in: *Journal of the Royal Statistical Society: Series B* 67.2 (2005), pp. 301–320.

[16] Leo Breiman, „Random Forests", in: *Machine Learning* 45.1 (2001), pp. 5–32.

[17] Jerome H. Friedman, „Greedy Function Approximation: A Gradient Boosting Machine", in: *The Annals of Statistics* 29.5 (2001), pp. 1189–1232.

[18] Tianqi Chen and Carlos Guestrin, „XGBoost: A Scalable Tree Boosting System", in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[19] Francis X. Diebold and Roberto S. Mariano, „Comparing Predictive Accuracy", in: *Journal of Business & Economic Statistics* 13.3 (1995), pp. 253–263.

[20] yfinance developers, *yfinance Documentation*, 2026, `https://ranaroussi.github.io/yfinance/` (visited on 03/10/2026).

[21] Todd E. Clark and Kenneth D. West, „Approximately Normal Tests for Equal Predictive Accuracy in Nested Models", in: *Journal of Econometrics* 138.1 (2007), pp. 291–311.

[22] Raffaella Giacomini and Halbert White, „Tests of Conditional Predictive Ability", in: *Econometrica* 74.6 (2006), pp. 1545–1578.

[23] John C. Platt, „Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods", in: *Advances in Large Margin Classifiers*, ed. by Alexander J. Smola et al., MIT Press, 1999, pp. 61–74.

[24] Alexandru Niculescu-Mizil and Rich Caruana, „Predicting Good Probabilities with Supervised Learning", in: *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, 2005, pp. 625–632.

[25] Ivo Welch and Amit Goyal, „A Comprehensive Look at the Empirical Performance of Equity Premium Prediction", in: *The Review of Financial Studies* 21.4 (2008), pp. 1455–1508.

[26] Jacob Mincer and Victor Zarnowitz, „The Evaluation of Economic Forecasts", in: *Economic Forecasts and Expectations*, ed. by Jacob Mincer, National Bureau of Economic Research, 1969, pp. 1–46.

[27] Greta M. Ljung and George E. P. Box, „On a measure of lack of fit in time series models", in: *Biometrika* 65.2 (1978), pp. 297–303.

[28] Robert F. Engle, „Autoregressive Conditional Heteroscedasticity with Estimates of the Variance of United Kingdom Inflation", in: *Econometrica* 50.4 (1982), pp. 987–1007.

[29] David Harvey, Stephen Leybourne, and Paul Newbold, „Testing the equality of prediction mean squared errors", in: *International Journal of Forecasting* 13.2 (1997), pp. 281–291.

[30] Fakultet inženjerskih nauka Univerziteta u Kragujevcu, *Master radovi*, 2026, `https://www.cis.kg.ac.rs/master/` (visited on 03/10/2026).

[31] elektrotehnika, *finlatex: finthesis dokument klasa*, 2026, `https://github.com/elektrotehnika/finlatex` (visited on 03/10/2026).

[32] scikit-learn developers, *scikit-learn User Guide*, 2026, `https://scikit-learn.org/stable/user_guide.html` (visited on 03/10/2026).

[33] XGBoost developers, *XGBoost Documentation*, 2026, `https://xgboost.readthedocs.io/en/stable/` (visited on 03/10/2026).

[34] arch developers, *arch Documentation*, 2026, `https://bashtage.github.io/arch/` (visited on 03/10/2026).

[35] Marcos Lopez de Prado, *Advances in Financial Machine Learning*, Wiley, 2018.

Објављени радови

Кандидат није објавио радове током израде мастер рада.